

Fedokawaii — learn

Every key, every program, and two languages for the age of AI: from your first command to a small language model of your own

October 2026 · Fedora 44 · Python 3.14 · Rust · ROCm 7.2

目 Contents

案 How to read this book 4

一 Part I — From zero 6

始 1. What the pieces are 7
1.1 Files and folders 7
1.2 Paths 7
1.3 Programs and commands 7
1.4 Variables 8
1.5 Text, and why it matters here 8

端 2. Your first ten minutes in the terminal 9
2.1 Open it 9
2.2 Five commands, one at a time 9
2.3 The keys that save the most time 10
2.4 Help is always one command away 10

誤 3. Reading errors 11
3.1 The method: last line first 11
3.2 The ten you will meet most 11
3.3 Ask for an explanation 12

二 Part II — The desktop 13

窓 4. Keys, windows and layouts 14
4.1 The ten keys to learn first 14
4.2 How the windows arrange themselves 14
4.3 Workspaces: nine desks 14
4.4 The session 15

品 5. The bar, menus and notices 16
5.1 The bar, left to right 16
5.2 The menus 17
5.3 Notices 17

写 6. Clipboard, screenshots and recordings 19
6.1 The clipboard and its history 19
6.2 Screenshots 19
6.3 Recording the screen 19
6.4 Smaller pictures and PDFs: Super+Z 20
6.5 Archives and PDFs: Super+D 20

起 7. Starting things: launcher, calculator, downloads 21
7.1 The launcher 21
7.2 The calculator 21
7.3 The downloads centre 21

守 8. Looking after it: Super+U 23
8.1 The entries you will use 23
8.2 Snapshots: an undo for the whole system 23
8.3 The health report 24

三 Part III — The programs 25

言 9. bash and ble.sh: the shell 26
9.1 The moves 26
9.2 Use cases 26

探 10. Finding anything: fzf, fd, ripgrep 28
10.1 The moves 28
10.2 Use cases 28

歩 11. Moving and reading: zoxide, eza, bat, tldr 29
11.1 The moves 29

書 12. yazi: the files 30
12.1 The moves 30
12.2 Use cases 31

筆 13. neovim: the editor 32
13.1 Never stuck 32
13.2 The moves 33
13.3 Use cases 33

史 14. git, lazygit and GitHub 34
14.1 Once 34
14.2 The moves in lazygit 35
14.3 GitHub: gh 35
14.4 Use cases 35

動 15. btop: the machine 36

音 16. mpv, yt-dlp and Syncplay: video and sound 37

網 17. Brave: the web 39

遊 18. Steam: games 40

流 19. torrra: torrents 41

記 20. Notes, books and the small tools 42
20.1 PDFs: one tool for each job 42

問 21. Claude Code and herdr: AI agents 44

箱 22. podman: containers 45

四 Part IV — Python, for the age of AI 46

選 23. Why Python and Rust 47

環 24. The daily development loop 48
24.1 The tools, and what each does 48
24.2 A new project 48
24.3 Edit in neovim 49
24.4 Test on every save 49
24.5 Debug: look inside a running program 49
24.6 Commit and push 49
24.7 The whole loop, on one card 50

蛇 25. Python from zero 51
25.1 Values and variables 51
25.2 Text 51
25.3 Lists and loops 52
25.4 Dictionaries 52
25.5 Decisions 52
25.6 Functions, and your first real test 52
25.7 Files 53
25.8 Errors you expect: exceptions 53
25.9 Classes: data with behaviour 54
25.10 A command-line program 54

楽 26. Small fun projects 56
26.1 Your writing, as a chart 56
26.2 Flashcards 58
26.3 The weather, from a web API 59

26.4 The Game of Life 59

脳 27. A neural network from scratch, in NumPy 61
27.1 Arrays 61
27.2 What a network computes 61
27.3 The whole network, in 40 lines 62
27.4 Check the calculus 63
27.5 Handwritten digits 63

火 28. PyTorch on your graphics card 65
28.1 Where things stand (October 2026) 65
28.2 Install 65
28.3 Is the card there? 66
28.4 The digits network, in PyTorch 66

語 29. A tiny transformer that writes 69
29.1 The ideas, in plain words 69
29.2 The model 70

調 30. Fine-tuning a small model on your own data 73
30.1 The data: questions and answers about your desktop 73
30.2 The code 73

使 31. An agent with the Claude API 76
31.1 Set up 76
31.2 One question 76
31.3 The agent 77

五 Part V — Rust, for speed and safety 80

錆 32. Rust from zero: cargo and the loop 81
32.1 A project 81
32.2 In neovim 81
32.3 Check on every save 82
32.4 Reading the compiler 82

型 33. Rust by doing 83
33.1 Values 83
33.2 Ownership and borrowing 83
33.3 Vectors, maps and iterators 84
33.4 Structs and methods 84
33.5 Enums and match 84
33.6 Errors: Option, Result and ? 85
33.7 Traits and tests 85

具 34. A real command-line tool 87

微 35. A tiny autograd, in Rust 89

推 36. Running models in Rust: candle and burn 93
36.1 candle: the PyTorch model, in Rust 93
36.2 burn: tensors on the card, through Vulkan 94

橋 37. Rust inside Python: PyO3 95

道 38. Twelve weeks: a plan 97

辞 A. Glossary 98



How to read this book

This book is the practical one. It teaches by doing: a short example, what you should see, then an exercise of your own. The full handbook — how every part of the desktop is built and why — stays in the project's repository on GitHub (`docs/fedokawaii-guide.md`), for the day you want to know how something works inside. This book is about *using* it.

It has five parts. Read Part I first if you have never typed a command. After that, go where you need to: each chapter stands on its own and says what it expects you to know.

Part	You learn	Chapters
I — From zero	what a terminal, a file, a path, a program and a variable are; how to read an error	1–3
II — The desktop	the keys, windows, the bar and menus, the clipboard, screenshots, updates, snapshots	4–8
III — The programs	each program, its best moves, real uses and exercises	9–22
IV — Python	a daily development routine, then Python from zero up to a transformer, fine-tuning and an agent	23–31
V — Rust	Rust from zero up to your own autograd, inference in Rust, and Rust inside Python	32–38

How a lesson looks

Every new idea comes as an example first: you type it, then you see what it printed.

```
$ echo hello
hello
```

(the `$` is the prompt — do not type it; the line under it is what you should see)

A line starting with `$` is what you type, without the `$`. The lines under it are what the computer answers. Your answer may differ in details (a name, a date, a number): what matters is its shape. A block without `$` is a file's contents, to type into an editor.

The boxes

`Word — prompt.` The text the terminal shows when it waits for your command. Each new word gets a box like this the first time it appears.

Checkpoint. A short test at the end of a section. If you can do it without looking back, go on. If not, read the section again: it costs five minutes now and saves an hour later.

Careful. Something that can lose work or break a setting. Read it before you type.

The exercises

Each chapter ends with exercises, graded by stars:

Stars	Means	Takes
★	do exactly what the chapter showed, on your own files	a minute
★★	combine two things from the chapter	five minutes
★★★	a small project: you decide how	half an hour or more

Every exercise says what you should see when it works. Keep a line for each one you finish in your notes (`Super+N`): `- [x] 10.3 every PDF I own`. Watching that list grow is half the fun.

Three rules that make learning fast

1. **Type, never paste.** Your fingers learn the shapes: brackets, quotes, the order of words.
2. **One small step, then check.** Run it after every change. A mistake found at once is a mistake you understand.

3. **Read the error.** The last line of an error message almost always says what is wrong (chapter 3 shows how).



Part I — From zero

Never typed a command? Start here. Three short chapters: what the pieces are called, your first ten minutes in the terminal, and how to read the computer's complaints.

始 1. What the pieces are

In this chapter

- what a file, a folder, a path, a program, a command and a variable are
- one example of each, on this computer

You need: nothing.

Everything on a computer is one of a few kinds of thing. Once you can name them, every manual becomes readable.

始 1.1 Files and folders

A **file** is a named piece of stored data: a photo, a song, a letter, a program. A **folder** (also called a *directory*) holds files and other folders. Your own files live in your **home folder**, `/home/you` (with your user name instead of `you`), written `~` for short.

Thing	Example on this desktop	What it is
a file	`~/Notes/notes.md`	your notes: plain text
a folder	`~/Pictures/Screenshots`	where screenshots land
your home	`~` = `/home/you`	everything that is yours
the top of everything	`/`	every file on the computer sits somewhere below it

始 1.2 Paths

A **path** is a file's address: the folders you walk through to reach it, separated by `/`.

```
/home/you/Notes/notes.md
 |  |     |    |      └──── the file
 |  |     |    └─────── a folder inside your home
 |  |     └──────── your home folder
 |  └─────── the folder that holds every home
 └──────── the top: the root
```

`Word — absolute and relative path.` An *absolute* path starts at `/` (or `~`) and works from anywhere. A *relative* path starts where you are: standing in your home, `Notes/notes.md` means `/home/you/Notes/notes.md`. `..` means "one folder up", `.` means "here".

始 1.3 Programs and commands

A **program** is a file the computer can run. `nvim` (the editor), `mpv` (the video player) and `ls` (lists files) are programs. A **command** is a line you type to run one, often with **arguments** — words that tell it what to work on — and **options**, which start with `-`.

```
ls  -l  ~/Pictures
 |    |    └──── an argument: which folder
 |    └──── an option: the long listing
 └──── the program
```

始 1.4 Variables

A **variable** is a name that holds a value. The shell has some already: `$HOME` holds your home's path, `$USER` your name. In programming (Part IV), variables are the first thing you make.

```
$ echo $HOME
/home/you
$ name=Kyoto
$ echo "I would like to visit $name"
I would like to visit Kyoto
```

(no spaces around `=`: `name = Kyoto` would run a program called `name`)

始 1.5 Text, and why it matters here

Almost everything on this desktop is plain text: your notes, every setting, every script. Plain text is readable by any program, searchable (`rg`, chapter 10), and its history can be kept (git, chapter 14). That is why this book keeps returning to the terminal and the editor: they are the two tools that work on everything.

Exercises

1.1 ★ Name the parts. In the command `cp -r ~/Pictures /run/media/stick`, which word is the program, which the option, which the arguments?

Expected: the program is `cp` (copy), `-r` is an option (recursive: folders too), the two paths are arguments: from where, to where.

1.2 ★ Absolute or relative? Which of these work from any folder: `~/Notes`, `Notes`, `/etc/hosts`, `../Music`?

Expected: `~/Notes` and `/etc/hosts` (absolute); the other two depend on where you stand.

端 2. Your first ten minutes in the terminal

In this chapter

- open a terminal and run your first commands
- move between folders, make and remove a folder
- get help on any command

You need: chapter 1.

端 2.1 Open it

Press `Super+Return` (Super is the key with the Windows logo). A terminal window opens: foot, the desktop's terminal. It shows a **prompt**, then a blinking cursor.

```
⛩  ~ ------------------------------------------------ ☾  08:52:07 PM
›
```

(the top line shows the folder you are in — `~`, your home — and the time; you type after the `›`. After a command that failed, the top line also shows a vermilion `×` and a number: chapter 3)

`Word — shell.` The program inside the terminal that reads your commands and runs them. Here it is **bash**. The terminal is the window; the shell is what talks to you inside it.

端 2.2 Five commands, one at a time

Type each line (without the `$`) and press `Enter`. After each one, compare with the answer.

```
$ pwd
/home/you
```

(print working directory: where you are)

```
$ ls
Books  Documents  Downloads  Music  Notes  Pictures  repos  Videos
```

(list: what is here; your list may differ)

```
$ mkdir practice
$ cd practice
$ pwd
/home/you/practice
```

(make a folder, change into it, check)

```
$ echo "my first file" > first.txt
$ ls
first.txt
$ cat first.txt
my first file
```

(`>` sends what echo prints into a file; `cat` shows a file)

```
$ cd ..
$ pwd
/home/you
```

(`..` is one folder up)

> `Checkpoint.` Without looking: make a folder `test2` in your home, go into it, make a file `hello.txt` holding `hi`, show it, go back home. Then delete it all: `rm -r ~/test2`.

端 2.3 The keys that save the most time

Keys	Does
`Tab`	completes a name: type `cd Doc` then Tab, it becomes `cd Documents/`
Up · **Down**	earlier commands; type the start first (`git`) and Up finds only those
`Ctrl+R`	search every command you ever ran (fzf): type a few letters, Enter
`→` or **End**	take the grey suggestion the shell offers as you type
`Ctrl+C`	stop the running command (it does *not* copy here)
`Ctrl+Shift+C` · `Ctrl+Shift+V`	copy · paste in the terminal
`Ctrl+L`	clear the screen
`Ctrl+Shift+N`	a new terminal, in this same folder

端 2.4 Help is always one command away

You want	Type
the five most useful examples of a command	`tldr tar`
its whole manual	`man tar` (`/word` searches, `n` the next match, `q` quits)
a short list of its options	`tar --help`
the desktop's keys	`Super+/`

```
$ tldr mkdir
  Create directories and set their permissions.
  - Create specific directories:
    mkdir path/to/directory1 path/to/directory2 ...
  - Create specific directories and their parents if needed:
    mkdir -p path/to/directory1 path/to/directory2 ...
```

(for example; shortened)

Exercises

2.1 ★ A small tree. Make `~/practice/a/b/c` in one command (`mkdir -p`), then go into `c` using Tab to complete each name.

Expected: the prompt ends in `c`; `pwd` prints `/home/you/practice/a/b/c`.

2.2 ★ History. Press `Ctrl+R`, type `mkdir`, and run the command you typed in 2.1 again.

Expected: no error (`-p` does not mind that the folders exist).

2.3 ★★ Count. `ls ~/Pictures/Screenshots | wc -l` counts your screenshots. Take one with `Print` (drag a box), then count again.

Expected: one more than before. `|` sends what `ls` prints into `wc -l`, which counts lines.

3. Reading errors

In this chapter

- read an error message from its last line up
- recognise the ten errors you will meet most

You need: chapter 2.

An error is not a failure: it is the computer telling you, quite precisely, what it could not do. Programmers spend much of their day reading them. The trick is to read in the right order.

3.1 The method: last line first

1. Read the **last line**. It names the problem.
2. Find the **file and line number** it mentions. Go there.
3. Only then read the rest, from the bottom up: it is the path that led there.

```
$ python3 hello.py
Traceback (most recent call last):
  File "/home/you/hello.py", line 3, in <module>
    print(nmae)
          ^^^^
NameError: name 'nmae' is not defined
```

(the last line says what: a name nobody defined; the line above says where: line 3)

3.2 The ten you will meet most

You see	It means	Do
`command not found`	no program by that name: a typo, or not installed	check the spelling; `tldr` the name
`No such file or directory`	the path is wrong	`ls` the folder; use Tab to complete paths
`Permission denied`	not yours to change	system files: `sudoedit FILE`; a script: `chmod +x FILE`
`Is a directory`	you gave a folder where a file was wanted	add the file's name
`SyntaxError` (Python)	the code is not well formed: a missing `:`, `)` or quote	look at the line named, *and the one before*
`IndentationError` (Python)	the spaces at a line's start do not match	use the editor's indent (4 spaces) everywhere
`NameError` (Python)	a name used before it was given a value, or misspelt	compare the spelling with where you made it
`ModuleNotFoundError` (Python)	a library is not in this project	`uv add NAME` (chapter 24)
`error[E0382]: borrow of moved value` (Rust)	a value was given away, then used again	chapter 33: borrow it with `&`
`mismatched types` (Rust)	a number where text was expected, or similar	read the `expected` ⋯ `found` ⋯ lines

`Careful.` A long red error is not worse than a short one. Python and Rust print the whole path of calls; only the top and the bottom usually matter.

3.3 Ask for an explanation

When an error makes no sense, copy it (select it in the terminal: selecting copies) and ask Claude Code in the same folder (chapter 21): *"explain this error and what I should look at — do not fix it"*. You learn more from the explanation than from a fix.

Exercises

3.1 ★ Break it on purpose. Run `cd Nowhere`, `lss`, and `cat ~/Pictures`. Read each message.

Expected: `No such file or directory`, `command not found`, `Is a directory`.

3.2 ★★ Find the line. Save this as `~/practice/oops.py` (in neovim: `nvim ~/practice/oops.py`, `i`, type, `Esc`, `:wq`), run `python3 ~/practice/oops.py`, and fix it from the message alone:

```
prices = [3, 5, 8]
total = sum(prices)
print("total:" total)
```

Expected: `SyntaxError: invalid syntax. Perhaps you forgot a comma?` on line 3. With the comma (`print("total:", total)`) it prints `total: 16`.



Part II — The desktop

Five chapters on the desktop itself: the keys and windows, the bar and its menus, copying and capturing, starting programs, and looking after the machine. Learn the keys of chapter 4 first: everything else is faster once your hands know them.

窓 4. Keys, windows and layouts

In this chapter

- open, move, resize and close windows from the keyboard
- use nine workspaces as nine desks
- choose a layout for the work in front of you

You need: chapter 2.

Word — compositor. The program that draws every window and decides where it goes. Here it is **dwl**: it *tiles* windows — places them side by side, never overlapping — so you never drag a window to make room.

窓 4.1 The ten keys to learn first

Keys	Does
`Super+Return`	a terminal
`Super+Space`	the launcher: type a program's name, Enter
`Super+B` · `Super+E`	the browser (Brave) · the files (yazi)
`Super+Q`	close the window
`Super+J` · `Super+K`	focus the next · previous window
`Super+Shift+Return`	move this window to the first, largest place
`Super+F`	fullscreen, and back
`Super+1…9`	go to workspace 1 to 9
`Super+Shift+1…9`	send this window to that workspace
`Super+/`	every key, searchable

窓 4.2 How the windows arrange themselves

The default layout is a **golden spiral**: the first window takes the largest place on the left, each next one a smaller place, winding inward. Three layouts, one key each:

Keys	Layout	Good for
`Super+S`	spiral (the default)	one main window and helpers: an editor, then a terminal, then a browser
`Super+T`	columns, left to right	comparing side by side: two files, two terminals
`Super+Y`	floating	windows you move and size yourself, as on Windows
`Super+Shift+T`	the tiling layout used before	

`Super+H` and `Super+L` make the first window narrower or wider. `Super+Shift+Space` floats one window on its own; **Super+drag** moves it, **Super+right-drag** resizes it.

窓 4.3 Workspaces: nine desks

`Word — workspace.` A separate screen of windows. Only one shows at a time; the others keep their windows exactly as you left them. The bar shows them as kanji numerals: 一 二 三 …

A habit that works: `1` for the browser, `2` for code (editor and terminal), `3` for files, `4` for chat and music, `9` for games. `Super+Tab` jumps back to the workspace you were on: with two desks, it flips between them. `Super+0` shows all at once.

With two monitors, `Super+,` and `Super+.` move the focus left and right, and `Super+Shift+,` · `Super+Shift+.` send the window there.



4.4 The session

Keys	Does
`Super+Escape`	lock the screen (asleep after 90 s)
`Super+Shift+E`	the power menu: lock, sleep, log out, restart, shut down
`Super+Shift+N`	dismiss every notice
`Alt+Shift`	switch the keyboard layout (us / ar)
`Ctrl+Alt+F2`…`F6`	a text console, should the desktop ever freeze (`Ctrl+Alt+F1` back)

Exercises

4.1 ★ Three windows. Open a terminal, then yazi (`Super+E`), then another terminal. Press `Super+J` five times and watch the vermilion frame move.

Expected: the frame visits each window in turn: the focus follows the order of the spiral.

4.2 ★ Promote. Focus the smallest window and press `Super+Shift+Return`.

Expected: it swaps into the largest place on the left.

4.3 ★★ A coding desk. On workspace 2, open neovim in a terminal (`nvim`) and a second terminal. Switch to columns (`Super+T`), back to the spiral (`Super+S`). Send the second terminal to workspace 3 (`Super+Shift+3`) and follow it (`Super+3`), then go back (`Super+Tab`).

Expected: the bar's 三 lights while it holds a window; Super+Tab returns you to 二.

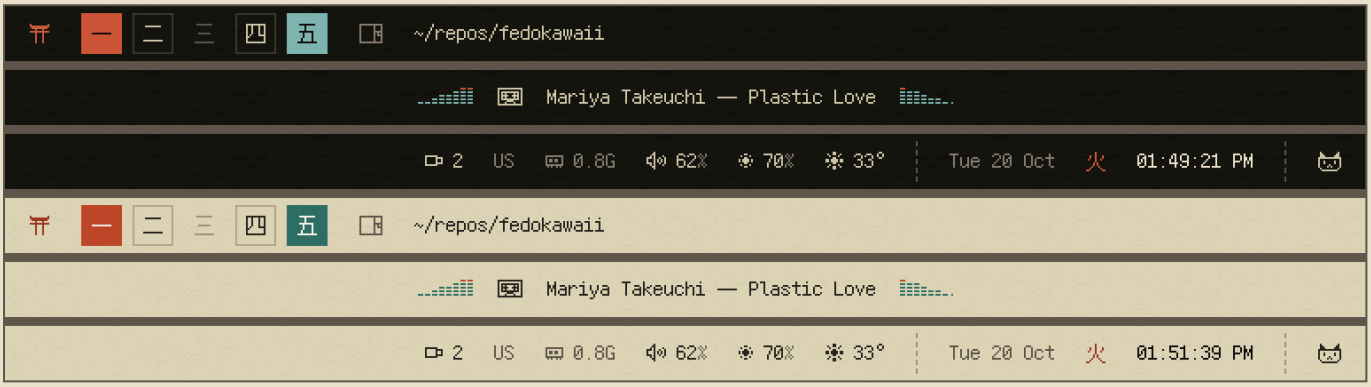
4.4 ★★★ A week without the mouse. For one week, open, move and close every window by key. Keep the help sheet (`Super+/`) a key away; write down the three keys you looked up most.

5. The bar, menus and notices

In this chapter

- read every part of the bar
- open quick settings, sound, calendar and power by key or click
- hold notices back while you work

You need: chapter 4.



The bar, its left, middle and right ends: dark theme above, light below.

5.1 The bar, left to right

Part	Shows	Click
the torii ⛩		the launcher
workspaces 一 二 三 …	the desks; 四–九 only while they hold a window	go there
the layout icon	spiral, columns or floating	switch spiral ⇄ columns
the title	the focused window	
now playing	the song, with a turning cassette and a level meter	play/pause; right: next
a seal 静 or 共有	do-not-disturb on · your screen is being shared	
the drive mark	a USB stick, disk or SD card is plugged in	open or eject it
the download mark	something is downloading: the percent of all of it	the downloads centre (chapter 7)
memory	how much the programs use (5.2G); vermilion past 85 %	btop
volume · brightness	speaker and sun, with the level	the sound panel · the brightness slider
weather	the sky and the temperature where you are	
date, day, time	the day as a kanji seal (月 Monday … 日 Sunday)	the calendar; the time: a timer
the cat	quick settings	quick settings



Quick settings, power, the calendar and the sound panel, on both themes.

5.2 The menus

Menu	Key	Inside
quick settings	`Super+X`	brightness and sound, theme, night light, network, Bluetooth, drives, backup, do-not-disturb
sound panel	`Super+A`	the volume, every output and input, the mixer
calendar	`Super+C`	the month; *next*, *today*
power	`Super+Shift+E`	lock · sleep · log out · restart · shut down
help	`Super+/`	every key; type to filter
wallpaper · theme	`Super+W` · `Super+Shift+W`	a picture for this theme · dark ⇄ light

Every menu works the same: arrows or `j` `k` move, `Enter` chooses, `Esc` closes, and typing filters the list.

5.3 Notices

A notice appears at the top right and fades by itself; a click does what it offers (opens the download, the drive). `Super+Shift+N` dismisses them all. While you need quiet — a call, an exam — turn on *dnd* in quick settings: notices wait, and **静** stands in the bar.

Exercises

5.1 ★ Every menu once. Open each menu of 5.2 by its key, then close it with `Esc`.

Expected: six menus, each in the same pixel style, each closed by Esc.

5.2 ★ Quiet. Turn on *dnd* (`Super+X`, *dnd*), then run `notify-send hello` in a terminal. Turn *dnd* off.

Expected: no notice while 静 is in the bar; the notice shows once dnd is off.

5.3 ★★ A 25-minute focus timer. Click the time in the bar and choose *25 min*.

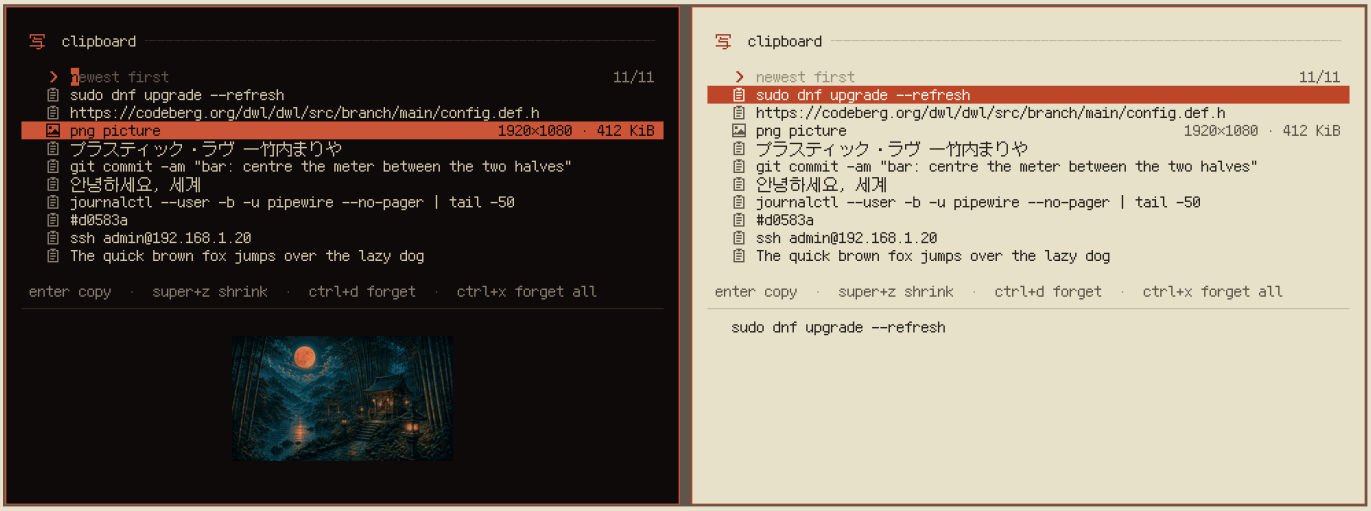
Expected: the time left shows in the timer's heading; a notice when it ends.

6. Clipboard, screenshots and recordings

In this chapter

- paste anything you copied earlier
- take, mark up and share a screenshot
- record the screen
- shrink a picture or PDF for email

You need: chapter 4.



The clipboard history (Super+V): text and pictures, filtered as you type.

6.1 The clipboard and its history

Where	Copy	Paste
every program	`Ctrl+C`	`Ctrl+V`
the terminal	`Ctrl+Shift+C`, or just select the text	`Ctrl+Shift+V`
anything copied earlier	—	`Super+V`, choose, `Enter`, then paste as usual

The history (`Super+V`) keeps text *and* pictures. Type to filter, `Ctrl+D` forgets one entry, `Ctrl+X` forgets them all.

6.2 Screenshots

Press	Takes	Then
`Print`	a region: drag a box (`Esc` gives up)	saved to `~/Pictures/Screenshots` and copied
`Shift+Print`	the whole screen	saved and copied
`Super+Print`	a region, opened in satty to mark it up	`Enter` copies, `Ctrl+S` saves

In satty: `z` an arrow, `r` a rectangle, `t` text, `m` numbered markers, `b` blur (for a password or a name), **1…5** the colour, `Ctrl+Z` undo.

6.3 Recording the screen

`Super+Shift+R` opens a small menu: the whole screen or a region, with the desktop's sound or without. Choose, and it records: 録 and the time gone by stand in the bar, in vermilion. `Super+Shift+R` again — or a click on 録 — stops it. A notice says where the file is (a click plays it), and its path is on the clipboard.

What	Where
the recordings	`~/Videos/Recordings/2026-10-20_14-05-31.mp4` (the date and time it started)
the format	MP4, the picture encoded by the graphics card (H.264), the sound AAC

Use it for a bug report, a short tutorial for a friend, or to watch your own coding session back.

6.4 Smaller pictures and PDFs: Super+Z

Copied a photo that is too big for an email? `Super+Z` offers: *email · under 1 MB*, *chat · under 500 KB*, a size you type, another format (JPEG, WebP, PNG), or a PDF at three sizes. The result is on the clipboard, ready to paste. In yazi, `Alt+S` does the same to selected files.

6.5 Archives and PDFs: Super+D

Copy a zip or a PDF and press `Super+D` (or select the files in yazi and press `Alt+F`). Only what fits the files you chose is offered: unpack an archive, or pack files into a zip, 7z or tar.zst at the compression you choose (with a password if you want). For PDFs: merge, split, take or delete pages, rotate them, take out the pictures or the text, make a scan searchable, lock it, draw on it or sign it. The result goes next to the original, and its path is copied to the clipboard.

Exercises

6.1 ★ Annotate. `Super+Print`, drag over this window, put an arrow (`z`) and a number marker (`m`), `Enter`. Paste it into a note (`Super+N` cannot hold pictures — paste it into a Brave chat or email instead).

Expected: the marked-up picture appears where you paste.

6.2 ★ The history. Copy three different words, one after another. Open `Super+V` and paste the first.

Expected: the first word, though you copied two others since.

6.3 ★★ Under 500 KB. Take a full screenshot (`Shift+Print`), then `Super+Z` → *chat*. Check both sizes: `ls -lh ~/Pictures/Screenshots | tail -2`.

Expected: the shrunk copy is under 500 KB.

6.4 ★★ A 20-second demo. Record yourself (`Super+Shift+R`, whole screen, no sound) opening the launcher, starting btop and closing it; stop, and click the notice.

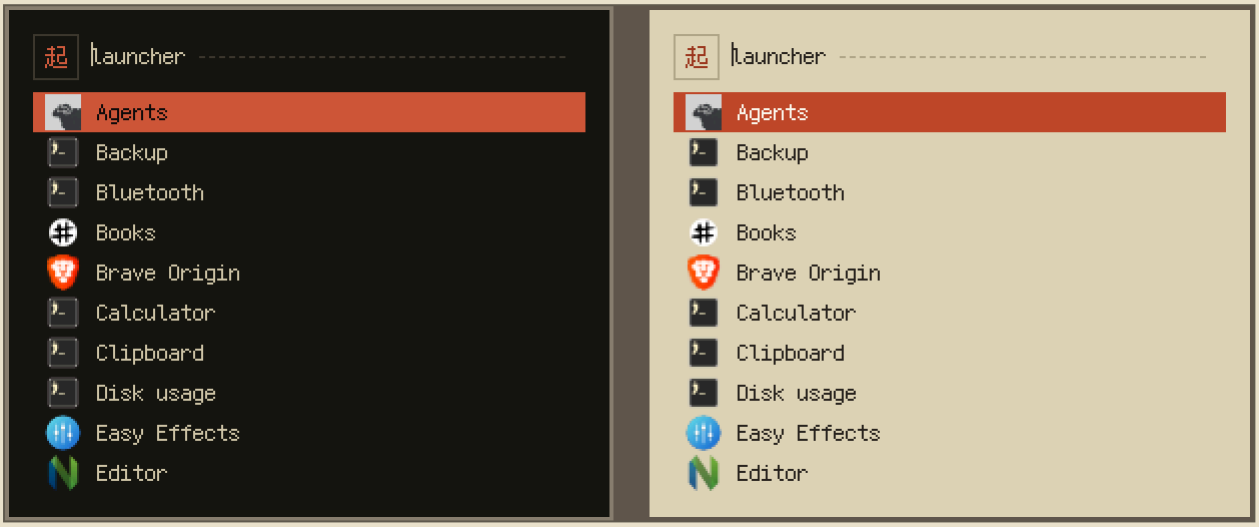
Expected: the recording plays in mpv; it is in `~/Videos/Recordings`.

起 7. Starting things: launcher, calculator, downloads

In this chapter

- start any program by a few letters of its name
- calculate without opening a program
- follow every download in one place

You need: chapter 4.



The launcher (Super+Space).

起 7.1 The launcher

Super+Space (or the torii). Type a few letters — bra for Brave, ste for Steam, syn for Syncplay — and Enter. The programs you start most rise to the top. Shift+Delete makes one forget its rank.

起 7.2 The calculator

Calculator in the launcher (Super+Space, calc): type a sum, a percentage or a conversion and the answer shows as you type; Enter copies it. ↑ brings back the sum before, Ctrl+U clears, Esc closes.

You type	You see
15% of 2400	360
120 usd to aed	the amount in dirhams (rates refreshed weekly)
5 ft to cm · 3.5 h to min	152.4 cm · 210 min
sqrt(2) · 255 to hex	1.414213562 · 0xFF

起 7.3 The downloads centre

Anything that downloads — Brave, Steam, a torrent — shows in the bar's download mark as one percent. Click it for the list: each download with its source, size and progress; a click opens where it lands. A notice says when each one finishes.

Exercises

7.1 ★ No mouse. Start Brave, yazi and btop from the launcher alone.

7.2 ★ Quick sums. In the calculator: how many seconds in a week (1 week to s)?

Expected: `604800 s`.

7.3 ★★ Watch it arrive. Download a Fedora image's checksum file in Brave (`https://fedoraproject.org`) and watch the bar's mark, then click the finished notice.

Expected: the mark appears while it downloads and goes when it is done; the notice opens `~/Downloads`.

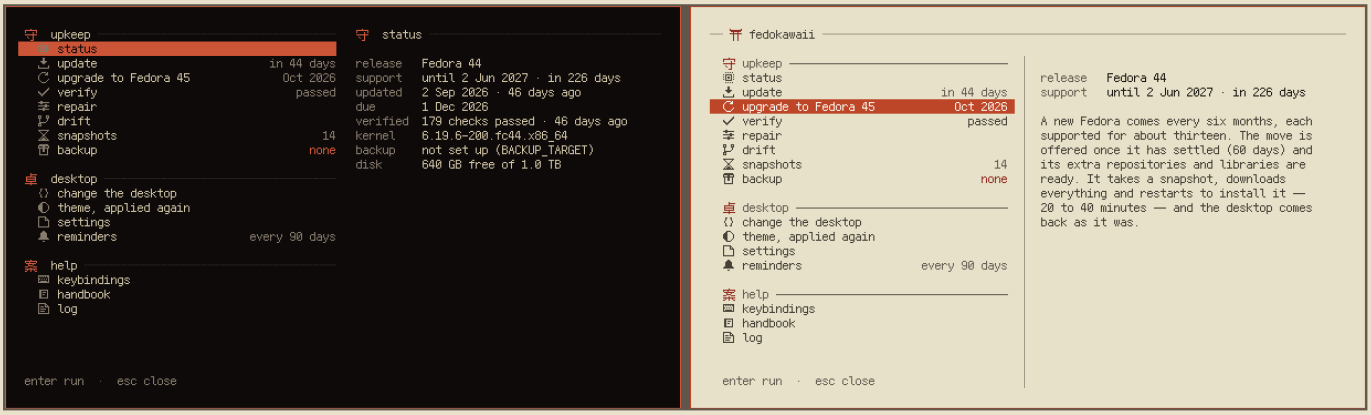
8. Looking after it: Super+U

In this chapter

- update the whole system in one step
- check its health, and repair what is wrong
- go back in time with snapshots

You need: chapter 4.

`Super+U` opens the system's own control: one window for everything that keeps the machine healthy. Each entry shows its state beside it — when the update is due, whether verify passed.



Super+U: the status on the dark theme, the move to the next Fedora on the light.

8.1 The entries you will use

Entry	Does	How often
status	how things stand, on one page	when curious
update	everything up to date: Fedora, Brave, Claude Code, the desktop's tools, neovim's plugins — with a snapshot first	each season, when the menu says *due*
update fedokawaii	the desktop itself, newest from GitHub	when you hear of a change
upgrade to Fedora N	the move to the next release	once it says *ready*
verify	every check; changes nothing	after something odd
repair	verify, then put right what it found	when verify fails
snapshots	the list of snapshots of the system	before and after a risky change
backup	your home to the backup drive	weekly
learn	this book	

From a terminal: `update`, `verify`, `repair`, `snaps`, and `fedokawaii report` (below).

8.2 Snapshots: an undo for the whole system

`Word — snapshot.` A frozen picture of every file at one moment. It costs almost no space (it shares unchanged data with the live files) and lets you copy any file back as it was.

Snapshots are taken by themselves: every hour, around every package install, before every update. Two everyday uses:

```
$ snaps
 # | Type   | Pre # | Date                     | Description
---+--------+-------+--------------------------+--------------
57 | single |       | Mon 20 Oct 2026 10:00:01 | timeline
58 | pre    |       | Mon 20 Oct 2026 10:12:40 | dnf install htop
59 | post   |    58 | Mon 20 Oct 2026 10:12:52 | dnf install htop
```

(for example; shortened)

You want	Do
undo one package install	`sudo snapper -c root undochange 58..59`, then reboot and `verify`
a file of yours as it was an hour ago	`sudo ls /home/.snapshots/` to find a number, then copy it back: `sudo cp -a /home/.snapshots/57/snapshot/$USER/Documents/file.ods ~/Documents/`
a mark before a risky change	`sudo snapper -c root create -p -d "before my experiment"` (prints its number)
everything back to that mark	`sudo snapper -c root undochange NUMBER..0`, then reboot

> Careful. `~/Downloads` is not in the snapshots: move what you keep out of it.

守 8.3 The health report

When something is wrong and you want help, `fedokawaii report` writes `~/fedokawaii-report.txt`: hundreds of checks, each with an ID and a verdict. Search it for `FAIL` (in neovim: `/FAIL`). Give that file to whoever helps you — or to Claude Code.

Exercises

8.1 ★ Status. Open `Super+U`, read *status*.

Expected: your Fedora release, until when it is supported, and when the next update is due.

8.2 ★ Verify. Run `verify` in a terminal.

Expected: a list of checks ending in *passed*, or the few to fix with what to do.

8.3 ★★ A safe experiment. Make a mark (`sudo snapper -c root create -p -d "before cowsay"`), install a toy (`sudo dnf install cowsay`), run `cowsay hi`, then put everything back to your mark (`sudo snapper -c root undochange NUMBER..0`, with the number the first command printed) and reboot.

Expected: after the reboot, `cowsay` answers `command not found`, and `verify` passes.

8.4 ★★ A lost file. Make `~/Documents/test.txt`, wait for the next hourly snapshot (or `sudo snapper -c home create -p`), delete the file, then copy it back from the snapshot.

Expected: `cat ~/Documents/test.txt` shows it again.



Part III — The programs

Each chapter here is one program, in the same order: what it is best at, the moves that unlock most of its power, real uses, then exercises. Learn a chapter's moves table by heart and you have most of the program; the rest is in its manual.

9. bash and ble.sh: the shell

In this chapter

- chain commands with pipes, and save output to files
- write your own commands: aliases and functions
- let ble.sh's colours catch mistakes before Enter

You need: chapter 2.

Best at: gluing programs together. Every program in this book can be run, combined and repeated from bash; a task you do twice by hand is a one-line command the third time.

9.1 The moves

Move	Example	Does
pipe `|`	`ls ~/Pictures | wc -l`	one program's output into the next
redirect `>` · `>>`	`date >> ~/log.txt`	output into a file: replace · append
`&&` · `||`	`mkdir x && cd x`	the second only if the first worked · only if it failed
`$( … )`	`echo "today is $(date +%A)"`	a command's output, in the middle of another
a loop	`for f in *.png; do echo "$f"; done`	the same command for each file
quotes	`"$f"`	keeps a name with spaces whole — quote every variable
`!!` · `Alt+.`	`sudo !!`	the last command · its last argument
an alias	`alias gs='git status'`	a short name for a long command
a function	`mkcd() { mkdir -p "$1" && cd "$1"; }`	a command of your own, with arguments
exit status	`echo $?`	0 means it worked; anything else, it failed

As you type, **ble.sh** colours the line: a known command in teal, an unknown one in vermilion, an existing file underlined. Vermilion before you press Enter is a typo caught for free. The grey suggestion is an earlier command: `→` takes it, `Ctrl+→` one word.

Your own aliases and functions go in `~/.bashrc.d/local.sh` — yours; the desktop never touches it. Every new terminal reads it.

9.2 Use cases

- rename a hundred photos by their date in one loop
- a `backup-notes` function that copies `~/Notes` to a USB stick
- `history | rg ffmpeg` to find that clever command from last month

Exercises

9.1 ★ mkcd. Add the `mkcd` function above to `~/.bashrc.d/local.sh`, open a new terminal, and run `mkcd ~/practice/new`.

Expected: the prompt shows `~/practice/new` at once.

9.2 ★★ Count by kind. How many files of each kind are in your pictures?

```
fd -t f . ~/Pictures | sed 's/.*\.//' | sort | uniq -c | sort -rn
```

Expected: lines such as `42 png`, the most common first. Read it as a sentence: list the files, keep what follows the last dot, sort, count each run, sort by the count.

9.3 ★★★ A daily line. Write a function `today` that prints the date, the weather line (`kawa-weather`), and how many lines your notes file has (`wc -l < ~/Notes/notes.md`).

探 10. Finding anything: fzf, fd, ripgrep

In this chapter

- find any file by a few letters of its name
- find any word inside any file, in a second
- pick anything from a list by typing

You need: chapter 9.

Best at: the question you ask a computer most — *where is it?* `fd` finds files by name, `rg` (ripgrep) finds text inside files, `fzf` turns any list into a searchable menu.

探 10.1 The moves

Command	Does
`fd notes`	files whose name contains *notes*, below here
`fd -e pdf . ~`	every PDF in your home
`fd -t d conf`	folders only
`fd -S +500m . ~`	files over 500 MB
`rg "rice" ~/Notes`	every line holding *rice*, with file and line number
`rg -i -t py "todo"`	ignore case, only Python files
`rg -l word` · `rg -C 2 word`	file names only · two lines of context around each match
`nvim "$(fzf)"`	pick a file by typing, edit it
`Ctrl+T` · `Alt+C` · `Ctrl+R`	insert a path · go into a folder · find an old command — all fzf

In every fzf list: words in any order narrow it; `'word` exact; `^start` and `end$`; `!not`.

探 10.2 Use cases

- `rg -i "invoice" ~/Documents -l` — which files mention invoices
- `fd -e mkv . ~/Videos -x du -h` — every film and its size
- in a code project, `rg "def train"` finds where a function is defined in a second

Exercises

10.1 ★ Every PDF you own. `fd -e pdf . ~`

Expected: a list of paths; nothing if you have no PDFs yet (try `-e png`).

10.2 ★ Grep your notes. `rg -i` a word you wrote in your notes.

Expected: `~/Notes/notes.md`, then the line number and the line.

10.3 ★★ Pick and edit. In your home, `nvim "$(fd -e md | fzf)"`.

Expected: a list of Markdown files only; Enter opens the one you chose.

10.4 ★★ The big ones. List the five largest files in your home: `fd -t f -S +100m . ~ -x du -h | sort -h | tail -5`.

10.5 ★★★ A finder of your own. Write a function `nf` (*note find*) that runs `rg -i "$1" ~/Notes` and, if nothing is found, says *nothing about $1*.

11. Moving and reading: zoxide, eza, bat, tldr

In this chapter

- jump to any folder by one word
- list folders with sizes, dates and git state
- read a file with colours

You need: chapter 2.

Best at: the small daily moves. `cd` here is **zoxide**: it remembers the folders you visit and jumps by a word. `ls`, `ll` and `lt` are **eza**. `cat` is **bat**.

11.1 The moves

Command	Does
`cd proj`	jump to the folder you use most whose name matches *proj*, from anywhere
`cdi`	the folders zoxide knows, in fzf
`ll`	everything, with sizes, dates and git state (`N` new, `M` modified)
`lt · eza -T -L 3`	a tree, two levels · three
`ll -s size -r · ll -s modified`	biggest first · newest last
`bat file.py`	the file with line numbers and syntax colours
`tldr ffmpeg`	the five commands you actually wanted
`gdu ~`	what fills the disk, folder by folder, with a delete key

Exercises

11.1 ★ Teach zoxide. `cd ~/Documents`, `cd /`, then `cd doc`.

Expected: you are back in `~/Documents`.

11.2 ★ A tree. `lt ~/repos` once you have a project (chapter 24).

11.3 ★★ Clean-up hunt. `gdu ~`: find the biggest folder in your home and decide whether it should stay. `q` quits.

12. yazi: the files

In this chapter

- move through folders with h j k l, preview anything
- copy, move, rename and delete many files at once
- jump anywhere by typing

You need: chapter 4.

Best at: working on many files fast, with a preview of each — pictures, PDFs, videos and code — without leaving the keyboard. `Super+E` opens it. It is also every program's file dialog here.



yazi: the parent folder, this folder, and a preview of the file under the cursor.

12.1 The moves

Keys	Does
`h` `j` `k` `l`	up a folder · down · up · open
`Space` · `v` · `Ctrl+A`	select one · a range · all
`y` · `x` · `p`	copy · cut · paste
`d` · **`D`**	to the trash · delete for good
`a`	new file (end with `/` for a folder)
`r`	rename; several selected: rename them all in neovim
`z` · **`Z`**	jump to anything below here (fzf) · to a folder you use (zoxide)

Keys	Does
`s` · **S**	search names (fd) · contents (ripgrep)
`.`	hidden files on / off
`g` `h` · `g` `d` · **M**	home · your drives · drives: open or eject
`Enter` on a film	it plays in mpv; **W**: watch it together (Syncplay)
`Alt+S`	shrink or convert pictures and PDFs
`Alt+F`	archives and PDFs: unpack, pack, merge, sign (Super+D's menu)
`;`	run a command on the selection: `%s` the selected files



12.2 Use cases

- sort a holiday's photos: select by range, cut, paste into a new folder
- rename 30 episode files at once with `r` and neovim's `:%s` (chapter 13)
- clean `~/Downloads`: sort by size (`,` then `s`), trash the big ones you watched

Exercises

12.1 ★ Three moves. In `~/practice`, make a folder `pics/` (`a`, `pics/`), copy two screenshots into it (`y`, then `p` inside it).

12.2 ★★ Rename many. Select three files, press `r`: in neovim, change all three names at once with `:%s/^/2026-/`, then `:wq`.

Expected: each name now starts with `2026-`.

12.3 ★★ Pack them. Select the files of 12.1, press `;`, type `7z a pics.7z %s`.

Expected: `pics.7z` appears in the folder.

筆 13. neovim: the editor

In this chapter

- edit without getting stuck: modes, save, quit, undo
- the grammar that makes editing fast: verb + motion
- LazyVim's Space menu: files, search, git, code

You need: chapter 2.

Best at: editing text — notes, settings, code — faster than any mouse, with the same keys everywhere: your notes window (`Super+N`), man pages and git messages all open in it. Here it runs **LazyVim**, a ready-made setup: a file finder, language servers, lazygit inside.



neovim with LazyVim: the file tree, a file, and the status line with the mode.

筆 13.1 Never stuck

Situation	Do
keys do odd things	`Esc`, twice: you are in Normal mode
save · save and quit · quit without saving	`:w` · `:wq` · `:q!` (or `Ctrl+S` to save)
undo · redo	`u` · `Ctrl+R`
recording @q at the bottom	press `q` once

> **Word — mode.** In *Normal* mode, keys are commands; in *Insert* mode (`i`), keys type text. `Esc` always goes back to Normal.

筆 13.2 The moves

Keys	Does
`i` · `a` · `o`	type before the cursor · after it · on a new line below
`w` · `b` · `0` · `$` · `gg` · `G`	next word · back a word · line start · end · first · last line
`s` and two letters	**flash**: jump anywhere visible by a label
`dd` · `yy` · `p`	cut · copy a line · paste
`ciw` · `ci"` · `dap`	change this word · inside the quotes · delete a paragraph
`.`	repeat the last change — the most useful key
`/word` · `n`	search · next match
`:%s/old/new/g`	replace everywhere in the file
`Space` `Space` · `Space` `/`	find a file · search text in the whole project
`Space` `e` · `Space` `g` `g`	the file tree · lazygit
`gd` · `K` · `Space` `c` `a` · `Space` `c` `r`	go to the definition · its documentation · a fix · rename everywhere

The grammar: a **verb** (`d` delete, `c` change, `y` copy) + a **motion** or **object** (`w` a word, `ip` a paragraph, `i(` inside brackets). `d3w` deletes three words; `yi"` copies what is inside the quotes. Learn five verbs and five objects, and you have twenty-five commands.

筆 13.3 Use cases

- edit any setting: `nvim ~/.bashrc.d/local.sh`
- write code with completion, errors underlined as you type, and a test runner (chapter 24)
- change a word everywhere in a project: `Space` `s` `r`

Exercises

13.1 ★ The tutor. Run `nvim +Tutor` and do lesson 1.

Expected: you can move, delete, and quit with `:q!` without thinking.

13.2 ★★ Fix a list. Make a file with five lines `tea`, `rice`, `tea`, `milk`, `tea`. Turn every *tea* into *coffee* with one command, then undo it, then do it with `*`, `ciw`, `n` and `.` instead.

Expected: the same result both ways; the second teaches you the dot.

13.3 ★★ Project search. In `~/repos/fedokawaii`, `Space` `/`, type `kawa-notes`.

Expected: every file that mentions it, with a preview; Enter jumps there.

13.4 ★★★ Thirty days. Use neovim for everything for a month. Each week, add one new move to your notes and use it on purpose all week.

史 14. git, lazygit and GitHub

In this chapter

- commit, branch and undo with lazygit
- put a project on GitHub with `gh`
- see who changed what, and when

You need: chapter 13.

Word — commit. A saved snapshot of a project, with a message saying why. **git** keeps every commit, so any version can come back. A **repository** is a project folder git looks after.

Best at: never losing work, and trying things without fear: commit, experiment, and if it goes wrong, go back. **lazygit** is git with a screen: `Space` `g` `g` in neovim, or `lazygit` in a terminal.



lazygit: the five panels on the left, the selected change on the right.

史 14.1 Once

```
$ git config --global user.name "Your Name"
$ git config --global user.email you@example.com
$ gh auth login
```

(`gh` asks a few questions: GitHub.com, HTTPS, log in with a web browser)

史 14.2 The moves in lazygit

Keys	Does
`1` … `5`	the panels: status, files, branches, commits, stash
`Space` · `a`	stage a file · all files
`Enter` on a file	its lines: stage only some of them
`c`	commit (type the message, Enter)
`P` · `p`	push · pull
`n` (branches) · `Space`	a new branch · switch to one
`z`	undo the last git action
`s`	stash the changes away for a moment
`?`	every key of this panel

史 14.3 GitHub: gh

lazygit does the work on your own machine; **gh** talks to GitHub.

Command	Does
`gh repo create NAME --private --source . --push`	put this project on GitHub
`gh repo clone USER/NAME`	a copy of a project
`gh pr create` · `gh pr list`	a pull request · the open ones

史 14.4 Use cases

- every project in `~/repos` is a repository: the prompt shows its branch and changes
- an experiment on a branch: if it works, merge; if not, delete the branch
- `git log -p FILE`: why a line is the way it is

Exercises

14.1 ★ First commit. `mkcd ~/repos/hello`, `git init`, make `README.md` with one line, commit it in lazygit (`a`, `c`).

Expected: lazygit's commits panel (`4`) shows one commit.

14.2 ★★ Undo. Change the README, commit; then undo that commit with `z` in lazygit.

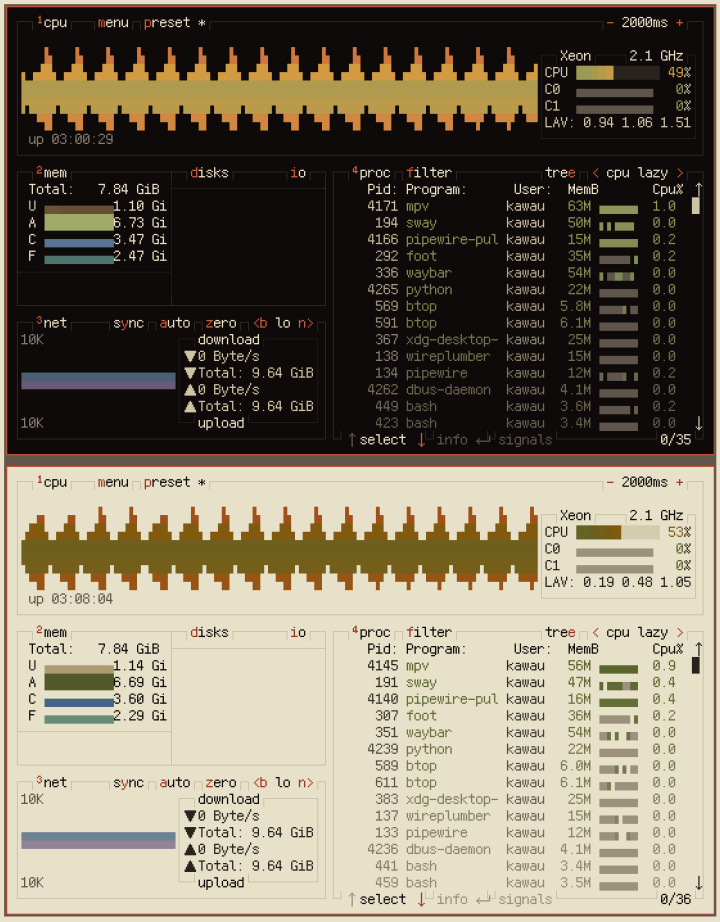
14.3 ★★ A branch. Make a branch `idea`, commit a new file there, switch back to `main`.

Expected: the file disappears on `main` and returns on `idea`.

14.4 ★★★ On GitHub. Create the repository on GitHub with `gh repo create`, push, and open it in Brave (`gh repo view --web`).

動 15. btop: the machine

Best at: seeing at once what the processor, memory, disks, network and graphics card are doing, and which program is to blame. Click the bar's memory, or run `btop`.



btop: the processor, memory, network and the process list.

Keys	Does
`1` · `2` · `3` · `4`	the processor, memory, network, process boxes on / off
`f`	filter the processes: type `brave`
`e`	show processes as a tree
`↑` `↓` then `t` · `k`	end a process politely · kill it at once
`p`	cycle layout presets
`m` · `o`	menu · options

On AMD it shows the graphics card's load, memory, heat and watts: watch it while a game or a training run (chapter 28) works.

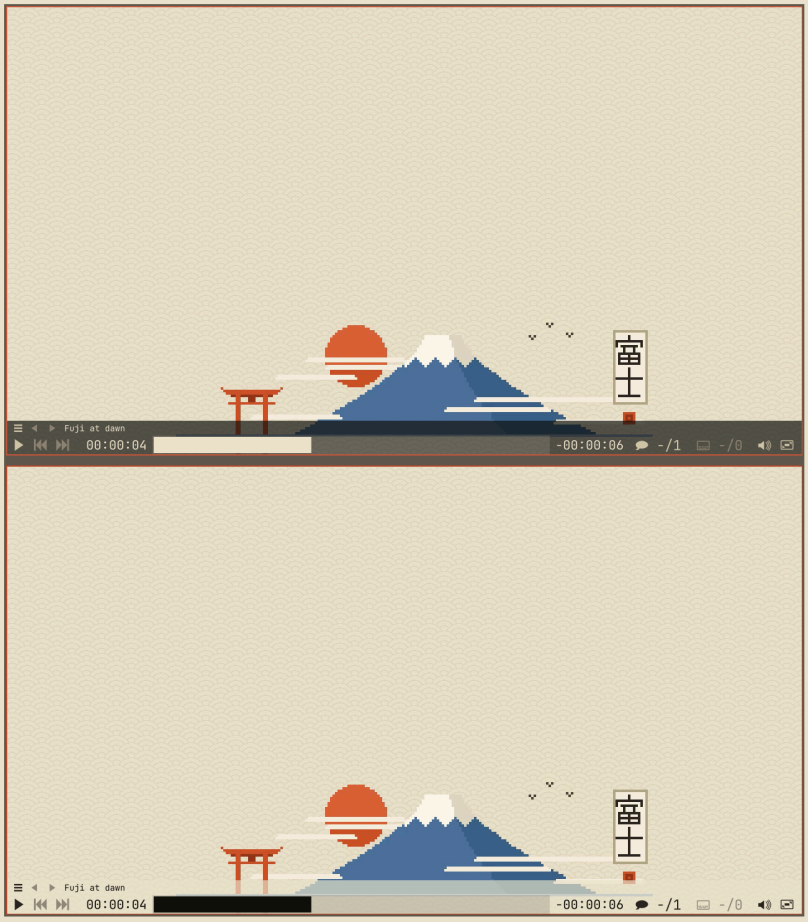
Exercises

15.1 ★ Who uses memory? Sort by memory (`←` `→` change the sorted column until it is *mem*, or click its heading) and name the top three programs.

15.2 ★★ Watch the card. Start a game or a video, then watch the GPU line rise.

音 16. mpv, yt-dlp and Syncplay: video and sound

Best at: playing anything — files, links, playlists — with the keyboard, and keeping your place. `mpv URL` plays a YouTube link directly (yt-dlp fetches it).



mpv, its controls shown on the picture.

Keys	Does
Space	pause
← → · ↑ ↓	5 s · 1 min
[ ] · Backspace	slower · faster · normal speed
v · j · #	subtitles on/off · next subtitle track · next audio track
z · Z	subtitles earlier · later
s	a screenshot of the frame
Ctrl+P	a menu of everything
q	quit, keeping the position

Command	Does
`mpv "https://youtu.be/…"`	play a web video
`yt-dlp "URL"`	keep it, in the best quality
`yt-dlp -x --audio-format opus "URL"`	the sound only
`mpv --speed=1.5 lecture.mp4`	a lecture at 1.5×

Syncplay: a film with a friend elsewhere, each from your own copy. In yazi, **W** on the film; agree on a room name; one person's pause pauses both. Enter in mpv chats.

Exercises

16.1 ★ A lecture faster. Play a long talk at 1.5× and slow it back with `Backspace`.

16.2 ★★ Keep the audio. Download a talk's sound as Opus and play it in mpv.

16.3 ★★ Together. Watch a short film with a friend through Syncplay.

網 17. Brave: the web

Best at: the web, privately, with ads and trackers blocked. `Super+B`.

Keys	Does
`Ctrl+L`	the address bar: type a search or an address
`Ctrl+T` · `Ctrl+W` · `Ctrl+Shift+T`	new tab · close · reopen the last closed
`Ctrl+Shift+A`	search your open tabs
`Ctrl+F` · `Ctrl+J` · `Ctrl+H`	find in page · downloads · history
`Ctrl+Shift+N`	private window
`F12`	the developer tools: a page's code, network, console

For the developer: the developer tools' *Console* runs JavaScript on any page; *Network* shows every request a page makes — the fastest way to learn how a website talks to its server.

Exercises

17.1 ★ Tab search. Open ten tabs, then find one with `Ctrl+Shift+A`.

17.2 ★★ Look inside. On any page, `F12` → *Network*, reload: which request is the largest?

遊 18. Steam: games

Best at: Windows games on Linux through Proton. The gamepad in the bar shows Steam is on; right-click it for its menu.

Launch option (right-click a game → Properties)	Does
`gamemoderun %command%`	the processor tuned for the game
`mangohud %command%`	the overlay: frames per second, temperatures (`Right Shift+F12`)
`gamescope -W 2560 -H 1440 -r 144 -f -- %command%`	the game in its own compositor, fixed size and rate

If a game will not start: Properties → Compatibility → force a Proton version (GE-Proton is installed). ProtonDB (`protondb.com`) lists what others did for each game.

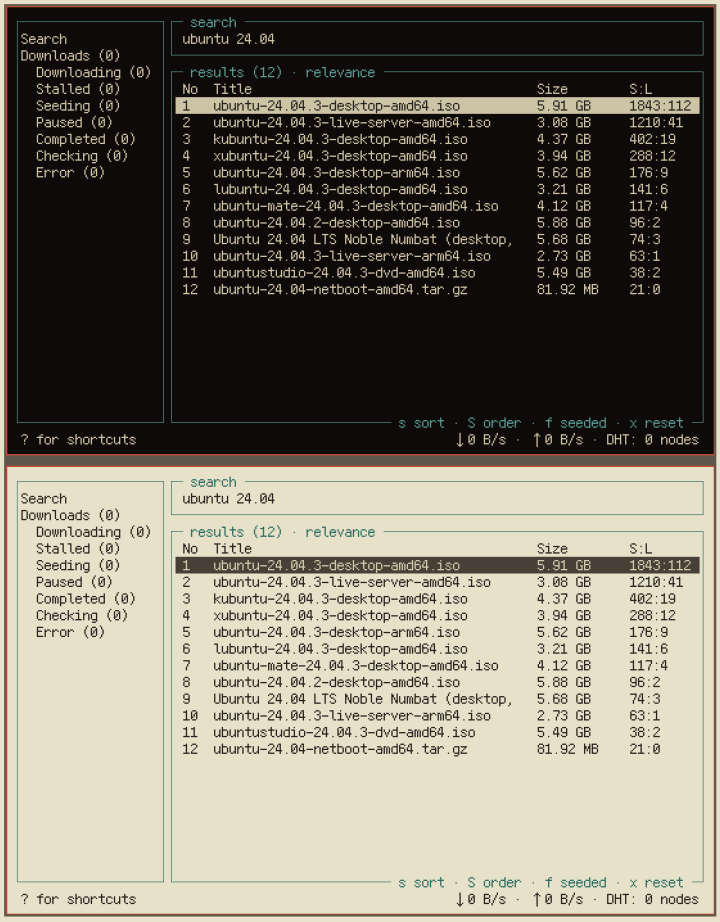
Exercises

18.1 ★ The overlay. Add `mangohud %command%` to a game and find its frame rate.

18.2 ★★ Compare. Play a minute with and without `gamemoderun` and compare the frame rate.

流 19. torrra: torrents

Best at: searching many torrent sites at once and downloading, in a terminal. *Torrents* in the launcher, or `torrra`. The first time, Prowlarr (its search service) asks for indexers.



torrra: one search across every indexer, seeders and size per result.

Keys	Does
type, Enter	search every indexer
j k	move
Enter	the torrent's details, then its files; Enter again downloads
s · f	sort menu · hide results nobody seeds

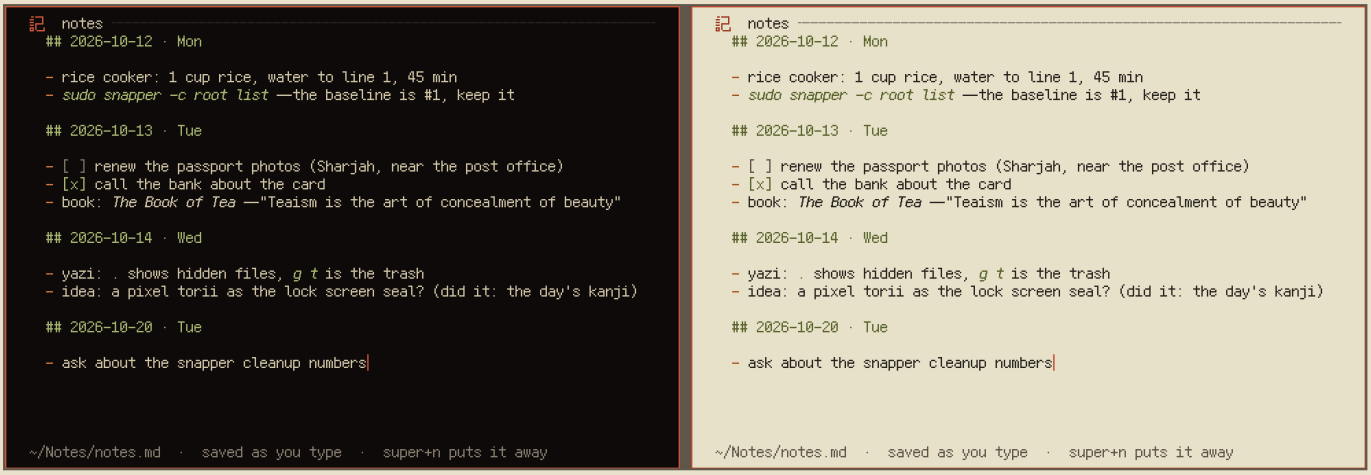
The `S:L` column is seeders:leechers: more seeders, faster. The bar's download mark follows it.

Exercises

19.1 ★ A legal one. Search `debian netinst` and download the image.

Expected: it appears in the downloads centre, then a notice when it is done.

20. Notes, books and the small tools



The notes window (Super+N): one file, a heading per day, ready to type.

Tool	Opens with	Best at
notes	`Super+N`	one Markdown file, a heading per day: type at once, saved as you type
books	`Super+R`	your PDFs, EPUBs and comics; `Ctrl+A` brings books in from Downloads
zathura	opening a PDF	reading; `Ctrl+R` recolours the page to the theme
VisiData	`vd file.csv`	tables: sort (`[ ]`), select rows by a pattern (`|`), sum, group (`Shift+F`)
jq	`jq . file.json`	reading and reshaping JSON
gdu	`gdu ~`	what fills the disk
LibreOffice	opening a .docx, .xlsx, .pptx	office files

20.1 PDFs: one tool for each job

Job	Tool	Command
merge	qpdf	`qpdf --empty --pages a.pdf b.pdf -- merged.pdf`
some pages only	qpdf	`qpdf in.pdf --pages . 3-5 -- part.pdf`
split into pages	qpdf	`qpdf --split-pages in.pdf page-%d.pdf`
rotate	qpdf	`qpdf in.pdf --rotate=+90:2 out.pdf` (page 2; leave out `:2` for all)
lock with a password	qpdf	`qpdf --encrypt --user-password=OPEN --owner-password=OWNER --bits=256 -- in.pdf locked.pdf`
the text out	pdftotext (poppler)	`pdftotext -layout in.pdf out.txt`
the pictures out	pdfimages (poppler)	`pdfimages -png in.pdf img` → `img-000.png`, `img-001.png` …
smaller	ghostscript	`gs -q -dNOPAUSE -dBATCH -dSAFER -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook -sOutputFile=small.pdf in.pdf`

(`/ebook` is 150 dpi pictures; `/screen` smaller, `/printer` sharper. Super+Z runs the same ghostscript on a PDF you copied, and Super+D (§6.5) runs the qpdf and poppler commands above)

Exercises

20.1 ★ A to-do. In your notes, write `- [ ] finish chapter 20`. Later, make it `- [x]`.

20.2 ★★ A table. Save a few expenses as `~/practice/money.csv` (`date,item,amount`), open it in VisiData, sort by amount, and find the total (on the amount column: **z+** then *sum*).

20.3 ★★ JSON. `curl -s "https://api.github.com/repos/neovim/neovim" | jq .stargazers_count`

Expected: one number: neovim's stars on GitHub.

20.4 ★★ A booklet. Take pages 1–3 of this guide (`docs/Fedokawaii-Learn.pdf` in your working copy) and pages 1–2 of another PDF, merge them into one file, rotate its second page, lock it with a password, and shrink it.

Expected: zathura asks for the password; `ls -l` shows the shrunk file smaller.

問 21. Claude Code and herdr: AI agents

`Word — agent.` A program that uses an AI model to work on a task in steps: it reads files, runs commands and edits, asking you first. **Claude Code** is one, in the terminal.

Best at: explaining code and errors, planning a change, writing tests, doing a well-described chore. You stay the author: you read every change before you keep it.

In a terminal	Does
`cd ~/repos/hello && claude`	start it in a project
type a request, `Enter`	it plans, asks, then works
`Esc`	stop it at once
`Shift+Tab`	switch modes: ask before edits · accept edits · plan only
`/clear` · `/init`	a fresh conversation · write a `CLAUDE.md` describing the project
`/exit`	leave

herdr runs several agents side by side, with a sidebar marking each *working*, *blocked* (it needs you) or *idle*. *Agents* in the launcher. Every herdr key is `Ctrl+B`, then one key: `Ctrl+B` `a` a new Claude, `Ctrl+B` `v` a split, **Ctrl+B h/j/k/l** move, `Ctrl+B` `o` the agent that just called, `Ctrl+B` `q` close the window (the agents keep working).

Habits that keep you learning:

1. Commit before you ask for a change: `git diff` then shows exactly what it did.
2. Ask it to *explain* first, *write* second: "explain what this function does, line by line".
3. Ask for tests with every change, and run them yourself.
4. Never paste a secret into a prompt.

Exercises

21.1 ★ Explain. In `~/repos/fedokawaii`, ask Claude Code: *"explain bin/kawa-weather to a beginner, step by step — do not change anything"*.

21.2 ★★ Two at once. In herdr, run one agent writing tests for your `hello` project while another explains git's reflog to you.

箱 22. podman: containers

> `Word — container.` A program packed with everything it needs, run apart from the rest of the system. Removing it leaves nothing behind.

Best at: trying software without installing it, and running services (Prowlarr runs in one here). podman runs containers as you, never as root.

Command	Does
`podman run --rm -it fedora:44 bash`	a throw-away Fedora, gone when you `exit`
`podman run --rm -it python:3.14 python`	a Python of another version, for a minute
`podman ps -a` · `podman images`	running and stopped containers · the images kept
`podman run --rm -v ~/practice:/w:Z -w /w python:3.14 python script.py`	run a script of yours inside
`podman image prune`	delete images no container uses

Exercises

22.1 ★ A throw-away system. Start the Fedora container, `dnf install -y cowsay`, run it, `exit`.

Expected: `cowsay` exists only inside; on your desktop, `command not found`.

22.2 ★★ Your script inside. Run `~/practice/oops.py` (fixed) in the `python:3.14` container with `-v`.



Part IV — Python, for the age of AI

From your first line of Python to a language model you trained yourself, an AI fine-tuned on your own notes, and an agent that uses tools. Every chapter builds on the one before; every step has something to run and something to see.

23. Why Python and Rust

In this chapter

- why these two languages, and not others
- the plan of Parts IV and V

You need: nothing.

The AI world runs on two layers. On top, people describe models, data and experiments — in **Python**: PyTorch, NumPy, Hugging Face, every research paper's code, the SDKs of Claude and the other models. Underneath, the parts that must be fast and must never crash — tokenizers, inference servers, data loaders, the tools that run AI on your own machine — are increasingly written in **Rust**: Hugging Face's `tokenizers` and `safetensors`, the `candle` and `burn` frameworks, `uv` (the Python tool you will use) and `ruff` are Rust.

	Python	Rust
you write	what to do, briefly	what to do, and exactly who owns each piece of memory
speed	slow itself; fast through libraries written in C, C++ and Rust	as fast as C
mistakes found	when the line runs	before it runs: the compiler refuses unsafe code
for AI	the language of training, research, data, agents	the language of fast, safe tools and inference
learn it	in weeks	in months — and it teaches you how computers really work

Together they cover the whole range: experiment in Python, make the slow part fast in Rust, call it from Python (chapter 37). Other good choices exist — TypeScript for the web, C++ for CUDA kernels, Mojo — but no other pair covers both research and production as well, today.

The plan. Chapter 24 sets up the daily routine once. Chapters 25–26 teach Python by small, tested programs. Chapters 27–31 climb: a neural network in NumPy alone, PyTorch on your graphics card, a transformer, fine-tuning, an agent. Part V does the same for Rust.

24. The daily development loop

In this chapter

- make a project with its own Python and libraries (uv)
- edit with completion and errors shown as you type (neovim)
- run, test on every save, debug with breakpoints
- commit and push (lazygit, GitHub)

You need: chapters 13 and 14.

Professional programmers repeat one loop all day: *edit → run → test → commit*. Set it up once here; every later chapter uses it.

24.1 The tools, and what each does

Tool	Job
uv	makes a project, picks its Python, installs its libraries into the project's own folder, runs it
neovim (LazyVim)	the editor: completion, errors as you type, formatting, tests, the debugger
pytest	runs your tests — from neovim, once or on every save
ruff	finds mistakes and formats the code (neovim runs it on save)
git, **lazygit**, **gh**	history, and GitHub

`Word — virtual environment (venv).` A folder (`.venv`) holding one project's Python and its libraries, apart from every other project's. uv makes and fills it; you never activate it by hand: `uv run` uses it.

24.2 A new project

```
$ cd ~/repos
$ uv init hello
Initialized project `hello` at `/home/you/repos/hello`
$ cd hello
$ ls -a
.  ..  .git  .gitignore  main.py  pyproject.toml  .python-version  README.md
$ uv run main.py
Hello from hello!
```

(uv made a git repository too, with a `.gitignore` that keeps `.venv` out of it)

File	Holds
`main.py`	your program
`pyproject.toml`	the project's name, its Python version and its libraries
`.python-version`	which Python this project uses
`uv.lock`	(after the first `uv add`) the exact version of every library: the same everywhere

Libraries come in with `uv add`; tools you only need while developing, with `--dev`:

```
$ uv add --dev pytest
Resolved 6 packages in 0.4s
Installed 5 packages in 12ms
```

環 24.3 Edit in neovim

```
$ nvim .
```

Python support is set up already: the language servers (basedpyright and ruff), formatting on save, the test runner and the debugger load with the first `.py` file. In one:

Keys	Does
typing	completion appears; `Enter` takes it, `Ctrl+Space` asks for it
`K` · `gd` · `gr`	the documentation of a name · its definition · everywhere it is used
`]d` · `Space` `c` `d`	the next problem · the full text of this line's problem
`Space` `c` `a` · `Space` `c` `r`	a fix the language server offers · rename everywhere
`Space` `c` `f`	format the file (also on every save)
`Space` `r` `f`	run this file (`uv run`: the project's `.venv`)
`Space` `r` `r` · `Space` `r` `c` · `Space` `r` `l`	a Python prompt (REPL) under the code · send it the cell (`# %%` lines part a file into cells) · the line
`Space` `t` `t` · `Space` `t` `r` · `Space` `t` `w`	run this file's tests · the test under the cursor · run them on every save
`Space` `t` `s` · `Space` `t` `o`	the test summary · the last test's output
`Space` `d` `b` · `Space` `d` `c`	a breakpoint here · run with the debugger
`Space` `t` `d`	debug the test under the cursor
`Space` `d` `i` · `Space` `d` `O` · `Space` `d` `o`	step into · over · out of
`Space` `d` `u` · `Space` `d` `t`	the debugger's panels · stop it
`Space` `c` `v`	choose the project's `.venv` (when a project has more than one)
`Space` `g` `g`	lazygit
`Ctrl+/`	a terminal at the bottom, and back

環 24.4 Test on every save

Open a test file and press `Space` `t` `w`: neovim now runs its tests every time you save (`Ctrl+S`), in a fraction of a second, and marks each test in the margin — ✓ green, ✗ red, the failure's message beside the line. `Space` `t` `s` keeps the summary open on the right. Green means your last change kept everything working. This loop — change, save, glance at the margin — is the fastest way to learn a language there is. `Space` `t` `w` again stops watching.

環 24.5 Debug: look inside a running program

When a result is wrong and you cannot see why, stop the program in the middle and look:

1. Cursor on a line inside the function, `Space` `d` `b`: a red dot in the margin.
2. `Space` `d` `c`: the program runs and stops there; the panels show every variable's value.
3. `Space` `d` `O` steps one line; watch the values change. `Space` `d` `t` stops.

環 24.6 Commit and push

`Space` `g` `g`, `a` (stage all), `c`, a message that says *why*, `Enter`, **P**. The first time, put the project on GitHub:

```
$ gh repo create hello --private --source . --push
✓ Created repository you/hello on GitHub
✓ Pushed commits to https://github.com/you/hello.git
```

環 24.7 The whole loop, on one card

Step	Command or keys
new project	`uv init NAME && cd NAME && uv add --dev pytest`
a library	`uv add numpy`
edit	`nvim .` · Space Space a file
run	Space r f
test on save	Space t w in a test file
debug	Space d b, Space d c
commit · push	Space g g, a, c, **P**

Exercises

24.1 ★ The loop once. Make the `hello` project, add pytest, write `tests/test_one.py` with `def test_one(): assert 1 + 1 == 2`, run `uv run pytest`.

Expected: `1 passed`.

24.2 ★★ Red, then green. Watch the test file (Space t w), change the test to `1 + 1 == 3`, save: see it fail, read the message, put it back.

Expected: the failure shows `assert 2 == 3` — pytest prints both sides.

24.3 ★★ A breakpoint. In `main.py`, make a variable, put a breakpoint on the next line, run the debugger, and read the variable's value in the panel.

蛇 25. Python from zero

In this chapter

- values, variables, text, lists, dictionaries
- decisions and loops
- functions, and a test for each
- files, errors, classes, and a small command-line program

You need: chapter 24.

Each section: an example to type, what you should see, then one change to try yourself. Type the short ones in Python's own prompt — `uv run python` in your project (`exit()` leaves) — and the longer ones in a file.

蛇 25.1 Values and variables

```
>>> 7 * 6
42
>>> price = 12.5
>>> count = 4
>>> price * count
50.0
>>> type(price), type(count)
(<class 'float'>, <class 'int'>)
```

(`>>>` is Python's prompt: type what follows it)

`Word — type.` The kind of a value: `int` (a whole number), `float` (a number with a point), `str` (text), `bool` (`True` or `False`), `list`, `dict`. The type decides what you can do with it: `"3" + "4"` is `"34"`, `3 + 4` is `7`.

Your turn: make `hours = 37.5` and `rate = 80`; print the week's pay.

蛇 25.2 Text

```
>>> name = "Kyoto"
>>> f"I would like to visit {name}"
'I would like to visit Kyoto'
>>> name.upper(), len(name), name[0], name[-1]
('KYOTO', 5, 'K', 'o')
>>> "rice, tea, milk".split(", ")
['rice', 'tea', 'milk']
```

(an f-string puts values into text: anything in `{ }` is worked out)

Your turn: from `"2026-10-20"`, get the year as a number: `int("2026-10-20".split("-")[0])`.

蛇 25.3 Lists and loops

```
>>> prices = [3, 5, 8]
>>> prices.append(13)
>>> prices
[3, 5, 8, 13]
>>> sum(prices), max(prices), prices[1:3]
(29, 13, [5, 8])
>>> for p in prices:
...     print(p, "→", p * 2)
...
3 → 6
5 → 10
8 → 16
13 → 26
>>> [p * 2 for p in prices if p > 4]
[10, 16, 26]
```

(the `...` lines are Python asking for the rest of the loop: press Enter on an empty one to run it. The last line is a list comprehension: a loop that builds a list, in one line)

> `Careful.` Python knows where a loop's body ends by its **indentation**: four spaces. The editor inserts them for you; never mix tabs and spaces.

蛇 25.4 Dictionaries

```
>>> stock = {"rice": 2, "tea": 0, "milk": 1}
>>> stock["rice"]
2
>>> stock["tea"] += 3
>>> [item for item, n in stock.items() if n == 0]
[]
>>> stock.get("bread", 0)
0
```

(a dictionary maps keys to values; `.get` gives a default instead of an error when a key is missing)

蛇 25.5 Decisions

```
temperature = 31

if temperature > 35:
    print("stay inside")
elif temperature > 25:
    print("shade and water")
else:
    print("a walk")
```

You should see: `shade and water`. Change the number and run it again until you have seen all three.

蛇 25.6 Functions, and your first real test

A function is a named piece of work: give it values, it gives back a result. Put this in `shop.py`:

```
def total(prices: list[float], discount: float = 0.0) -> float:
    """the sum of the prices, less a discount (0.1 is 10 %)"""
    return round(sum(prices) * (1 - discount), 2)
```

and this in `tests/test_shop.py`:

```
from shop import total


def test_total_without_discount():
    assert total([3, 5, 8]) == 16


def test_total_with_ten_percent_off():
    assert total([10, 10], discount=0.1) == 18
```

```
$ uv run pytest -q
..                                                       [100%]
2 passed in 0.02s
```

(the `: list[float]` *and* `-> float` *are type hints: they change nothing when it runs, but neovim uses them to warn you when you pass the wrong kind of value)*

`Checkpoint.` Add a test that an empty list costs `0`. Does it pass without changing `total`? (It should: `sum([])` is `0`.)



25.7 Files

```
from pathlib import Path

notes = Path.home() / "Notes" / "notes.md"
text = notes.read_text(encoding="utf-8")
lines = text.splitlines()
print(len(lines), "lines; the last one:", lines[-1])
```

You should see: the number of lines in your notes and the last line. `Path` joins folders with `/` and reads or writes a whole file in one call; `write_text` writes.



25.8 Errors you expect: exceptions

```
def parse_price(text: str) -> float:
    try:
        return float(text)
    except ValueError:
        raise ValueError(f"not a price: {text!r}") from None


print(parse_price("12.5"))
print(parse_price("twelve"))
```

You should see: `12.5`, then a traceback ending in `ValueError: not a price: 'twelve'`. Catch an error only where you can say something better about it; otherwise let it stop the program with its own message — that is what tells you where the bug is.

A test can check that an error comes:

```
import pytest
from prices import parse_price


def test_words_are_not_prices():
    with pytest.raises(ValueError, match="not a price"):
        parse_price("twelve")
```

蛇 25.9 Classes: data with behaviour

```
from dataclasses import dataclass


@dataclass
class Item:
    name: str
    price: float
    count: int = 1

    def cost(self) -> float:
        return self.price * self.count


basket = [Item("rice", 12.5, 2), Item("tea", 8)]
print(sum(i.cost() for i in basket))
print(basket[0])
```

You should see: `33.0`, then `Item(name='rice', price=12.5, count=2)` — a dataclass writes its own description.

蛇 25.10 A command-line program

```
"""greet — say hello, a number of times"""
import argparse


def main() -> None:
    parser = argparse.ArgumentParser(description="say hello")
    parser.add_argument("name")
    parser.add_argument("-n", "--times", type=int, default=1)
    args = parser.parse_args()
    for _ in range(args.times):
        print(f"hello, {args.name}")


if __name__ == "__main__":
    main()
```

```
$ uv run greet.py Kyoto -n 2
hello, Kyoto
hello, Kyoto
$ uv run greet.py --help
usage: greet.py [-h] [-n TIMES] name
```

(argparse writes the `--help` for you, and refuses a missing name with a clear message)

Exercises

25.1 ★ Pay. Write `pay(hours, rate)` with overtime: hours over 40 pay 1.5 × the rate. Three tests: 30 hours, 40 hours, 45 hours.

Expected: for a rate of 10: `300`, `400`, `475`.

25.2 ★ Vowels. `vowels(text)` counts a, e, i, o, u in any case. Test it with `"Fedora"`.

Expected: `3`.

25.3 ★★ Shopping list. Read a file of lines like `rice 2` into a dictionary `{"rice": 2}`; a line without a number counts as 1. Test it with a string, not a file (`text.splitlines()`).

25.4 ★★ FizzBuzz, tested first. Write the four tests (3 → `Fizz`, 5 → `Buzz`, 15 → `FizzBuzz`, 7 → `7`) *before* the function; watch them fail (`Space` `t` `w`), then make them pass.

25.5 ★★★ A to-do tool. `todo.py add TEXT`, `todo.py list`, `todo.py done N`, keeping the list in `~/Notes/todo.md` as `- [ ]` / `- [x]` lines. Test the parsing and the formatting apart from the file.

26. Small fun projects

In this chapter

- four programs you will actually use, each with tests that run in a blink

You need: chapter 25.

Each project is one file and one test file, in a project of its own (`uv init`, `uv add --dev pytest`). Watch the test file (`Space` `t` `w`) before you write the first line.

26.1 Your writing, as a chart

How much did you write in your notes each day? The notes file has a heading per day (`## 2026-10-16 · Fri`): count the lines under each.

```
"""notestats — how much you wrote in your notes, day by day"""
import re
import sys
from collections import Counter
from pathlib import Path

HEADING = re.compile(r"^## (\d{4}-\d{2}-\d{2})")


def lines_per_day(text: str) -> Counter:
    """count the non-empty lines under each day's heading"""
    counts = Counter()
    day = None
    for line in text.splitlines():
        if m := HEADING.match(line):
            day = m.group(1)
        elif day and line.strip():
            counts[day] += 1
    return counts


def bar(n: int, most: int, width: int = 30) -> str:
    return "█" * max(1, round(n / most * width))


def main() -> None:
    path = Path(sys.argv[1]) if len(sys.argv) > 1 else Path.home() / "Notes" / "notes.md"
    counts = lines_per_day(path.read_text(encoding="utf-8"))
    if not counts:
        print("no dated notes yet")
        return
    most = max(counts.values())
    for day, n in sorted(counts.items())[-14:]:
        print(f"{day}  {n:3}  {bar(n, most)}")


if __name__ == "__main__":
    main()
```

```
from notestats import bar, lines_per_day

NOTES = """## 2026-10-15 · Thu

buy rice
call the bank

## 2026-10-16 · Fri

- [ ] chapter 25
"""


def test_counts_lines_under_each_day():
    assert lines_per_day(NOTES) == {"2026-10-15": 2, "2026-10-16": 1}


def test_lines_before_any_heading_are_ignored():
    assert lines_per_day("hello\n## 2026-10-15 · Thu\nx\n") == {"2026-10-15": 1}


def test_bar_is_never_empty():
    assert bar(1, 100) == "█"
```

```
$ uv run notestats.py
2026-10-15    2  ████████████████████████████████████████
2026-10-16    1  ████████████████████
```

(for example) New here: a **regular expression** (`re`) finds the headings; `Counter` is a dictionary that counts; `:=` gives a name to a value inside a condition.

Extend it: the weekday you write most (`datetime.date.fromisoformat(day).strftime("%A")`).

楽 26.2 Flashcards

Learning Japanese or Arabic words? A file of `front = back` lines becomes a quiz.

```
"""flash — flashcards from a file of "front = back" lines"""
import random
import sys
from pathlib import Path


def load(text: str) -> list[tuple[str, str]]:
    cards = []
    for line in text.splitlines():
        if "=" in line and not line.startswith("#"):
            front, back = line.split("=", 1)
            cards.append((front.strip(), back.strip()))
    return cards


def check(answer: str, back: str) -> bool:
    return answer.strip().lower() == back.lower()


def main() -> None:
    cards = load(Path(sys.argv[1]).read_text(encoding="utf-8"))
    random.shuffle(cards)
    right = 0
    for front, back in cards:
        if check(input(f"{front} → "), back):
            right += 1
            print("  ✓")
        else:
            print(f"  × {back}")
    print(f"{right} of {len(cards)}")


if __name__ == "__main__":
    main()
```

```
$ uv run flash.py ~/Notes/japanese.txt
water → mizu
  ✓
book → hono
  × hon
1 of 2
```

Test it: `load` and `check` take strings, so their tests need no file and no keyboard. **Extend it:** keep the cards you missed in a second file and quiz those first next time — the idea behind spaced repetition.

26.3 The weather, from a web API

```
"""sky — the next hours' temperature, from Open-Meteo (no key needed)"""
import json
import urllib.request

URL = ("https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}"
       "&hourly=temperature_2m&forecast_hours=12&timezone=auto")


def hours(data: dict) -> list[tuple[str, float]]:
    h = data["hourly"]
    return [(t[-5:], c) for t, c in zip(h["time"], h["temperature_2m"])]


def main() -> None:
    with urllib.request.urlopen(URL.format(lat=25.35, lon=55.39), timeout=10) as r:
        data = json.load(r)
    for time, celsius in hours(data):
        print(f"{time}  {celsius:5.1f}°  {'▇' * round(celsius / 2)}")


if __name__ == "__main__":
    main()
```

(25.35, 55.39 is Sharjah: put your own place. The test feeds `hours` *a small dictionary shaped like the answer — no network in tests)*

26.4 The Game of Life

A grid of cells; each step, a live cell with two or three live neighbours lives on, a dead cell with exactly three comes alive. Simple rules, endless patterns.

```
"""life — Conway's Game of Life in the terminal"""
import time


def step(live: set[tuple[int, int]]) -> set[tuple[int, int]]:
    near: dict[tuple[int, int], int] = {}
    for x, y in live:
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                if dx or dy:
                    near[(x + dx, y + dy)] = near.get((x + dx, y + dy), 0) + 1
    return {c for c, n in near.items() if n == 3 or (n == 2 and c in live)}


def show(live: set[tuple[int, int]], w: int = 40, h: int = 20) -> str:
    return "\n".join("".join("▇▇" if (x, y) in live else "· " for x in range(w)) for y in range(h))


if __name__ == "__main__":
    cells = {(1, 0), (2, 1), (0, 2), (1, 2), (2, 2)}           # a glider
    for _ in range(60):
        print("\033[H\033[J" + show(cells))
        cells = step(cells)
        time.sleep(0.1)
```

Test it: a *blinker* — three cells in a row — turns into three in a column and back: `assert step({(0, 1), (1, 1), (2, 1)}) == {(1, 0), (1, 1), (1, 2)}`.

Exercises

26.1 ★ Run all four. Each with its tests green in the margin.

26.2 ★★ Weekday. Extend `notestats` with `--weekday`: lines per weekday.

26.3 ★★ Missed first. Extend `flash` as 26.2 suggests.

26.4 ★★★ Rain alert. Extend `sky` with `precipitation_probability` and print a warning if any of the next 12 hours is above 50 %. Then add it to your `today` function from 9.3.

27. A neural network from scratch, in NumPy

In this chapter

- arrays: many numbers worked on at once
- what a neural network computes: forward, loss, backward, update
- train one on a spiral and on handwritten digits, with no AI library at all

You need: chapter 25.

Before PyTorch does it for you, do it once by hand. Every model in the news — the largest language models included — learns by the four steps in this chapter, repeated billions of times.

```
$ uv init nn && cd nn
$ uv add numpy scikit-learn
$ uv add --dev pytest
```

27.1 Arrays

```
>>> import numpy as np
>>> a = np.array([1.0, 2.0, 3.0])
>>> a * 10
array([10., 20., 30.])
>>> M = np.array([[1, 2], [3, 4]])
>>> M.shape, M.T
((2, 2), array([[1, 3],
       [2, 4]]))
>>> M @ np.array([1, 1])
array([3, 7])
>>> M.sum(axis=0), M.max(axis=1)
(array([4, 6]), array([2, 4]))
```

(`@` multiplies matrices: each row of `M` times the vector, summed. `axis=0` works down the columns, `axis=1` along the rows)

`Word — tensor.` An array of any number of dimensions: a number (0), a list (1), a table (2), a stack of tables (3) … A batch of 64 colour pictures is a 4-dimensional tensor: 64 × 3 × height × width. Neural networks are made of tensor arithmetic.

Broadcasting stretches a smaller array to fit: a table (100 × 3) plus a row (3) adds the row to every line. It is how one line of NumPy does what a loop would do a hundred times — and fast, because the loop runs in C.

27.2 What a network computes

A two-layer network turns an input `x` (here 2 numbers: a point's position) into a score for each class (3: which arm of a spiral it is on):

```
h      = max(0, x @ W1 + b1)     a hidden layer: 64 numbers, negatives cut to 0 (ReLU)
scores = h @ W2 + b2             3 scores, one per class
p      = softmax(scores)         the scores as probabilities that add up to 1
loss   = -log(p[right class])    small when the right class got a high probability
```

Training is four steps, again and again:

Step	Does
forward	compute the scores and the loss for the data
backward	compute how the loss would change if each weight moved a little: its **gradient**
update	move every weight a small step against its gradient: downhill
repeat	until the loss stops falling

`Word — gradient.` For each weight, the slope of the loss: how much the loss rises when the weight rises a little. Step the other way and the loss falls. **Backpropagation** computes all the gradients at once with the chain rule, from the loss backwards.



27.3 The whole network, in 40 lines

```
# mlp.py — a two-layer neural network in NumPy alone: forward, loss, backward, update
import numpy as np

rng = np.random.default_rng(0)

def spiral(n=100, k=3):
    """n points in each of k arms of a spiral: X is (n*k, 2), y the arm of each point"""
    X, y = [], []
    for c in range(k):
        r = np.linspace(0.0, 1.0, n)
        t = np.linspace(c * 4, (c + 1) * 4, n) + rng.normal(0, 0.2, n)
        X.append(np.c_[r * np.sin(t), r * np.cos(t)])
        y.append(np.full(n, c))
    return np.vstack(X), np.concatenate(y)

X, y = spiral()
N, D, H, K = X.shape[0], 2, 64, 3

W1 = rng.normal(0, 1, (D, H)) * np.sqrt(2 / D); b1 = np.zeros(H)
W2 = rng.normal(0, 1, (H, K)) * np.sqrt(2 / H); b2 = np.zeros(K)
lr = 0.5

for step in range(3001):
    # forward
    h = np.maximum(0, X @ W1 + b1)                  # ReLU
    logits = h @ W2 + b2
    logits -= logits.max(axis=1, keepdims=True)     # (for numerical safety)
    p = np.exp(logits); p /= p.sum(axis=1, keepdims=True)
    loss = -np.log(p[np.arange(N), y]).mean()
    # backward: the gradient of the loss with respect to every number above
    dlogits = p.copy(); dlogits[np.arange(N), y] -= 1; dlogits /= N
    dW2 = h.T @ dlogits; db2 = dlogits.sum(axis=0)
    dh = dlogits @ W2.T; dh[h <= 0] = 0
    dW1 = X.T @ dh; db1 = dh.sum(axis=0)
    # update: a small step downhill
    for P, G in ((W1, dW1), (b1, db1), (W2, dW2), (b2, db2)):
        P -= lr * G
    if step % 500 == 0:
        acc = (p.argmax(axis=1) == y).mean()
        print(f"step {step:4d}  loss {loss:.3f}  accuracy {acc:.2f}")
```

```
$ uv run mlp.py
step    0  loss 1.033  accuracy 0.29
step  500  loss 0.116  accuracy 0.98
step 1000  loss 0.075  accuracy 0.99
step 1500  loss 0.059  accuracy 0.99
step 2000  loss 0.050  accuracy 0.99
step 2500  loss 0.044  accuracy 0.99
step 3000  loss 0.040  accuracy 0.99
```

At step 0 it guesses (29 %: about one in three); by step 500 it has learnt the spiral.

27.4 Check the calculus

Backward is where mistakes hide. Check any gradient by *nudging*: move one weight by a tiny `eps` up and down, and see how the loss changes. The two numbers must agree.

```
import numpy as np
rng = np.random.default_rng(1)
X = rng.normal(size=(5, 3)); y = np.array([0, 2, 1, 2, 0]); W = rng.normal(size=(3, 3))

def loss_and_grad(W):
    z = X @ W; z -= z.max(axis=1, keepdims=True)
    p = np.exp(z); p /= p.sum(axis=1, keepdims=True)
    loss = -np.log(p[np.arange(5), y]).mean()
    d = p.copy(); d[np.arange(5), y] -= 1
    return loss, X.T @ d / 5

loss, G = loss_and_grad(W)
eps = 1e-5
W[1, 2] += eps; up, _ = loss_and_grad(W)
W[1, 2] -= 2 * eps; down, _ = loss_and_grad(W)
print("by calculus", G[1, 2], " by nudging", (up - down) / (2 * eps))
```

```
by calculus 0.036057134325191066  by nudging 0.03605713432275337
```

(equal to nine digits: the backward pass is right)

27.5 Handwritten digits

scikit-learn ships 1 797 small pictures of handwritten digits, 8 × 8 pixels each:

```
from sklearn.datasets import load_digits
d = load_digits()
X, y = d.data / 16.0, d.target          # 64 numbers from 0 to 1 per picture; the digit
print(X.shape, y[:10])
```

```
(1797, 64) [0 1 2 3 4 5 6 7 8 9]
```

Exercises

27.1 ★ Too simple. Remove the hidden layer (scores straight from `X @ W + b`). What accuracy does the spiral reach now?

Expected: about 0.5: a straight line cannot separate a spiral. The hidden layer is what lets a network bend.

27.2 ★ Learning rate. Try `lr = 5.0` and `lr = 0.01`.

Expected: 5.0 jumps about or blows up (`nan`); 0.01 learns, but slowly. Choosing it is half the craft.

27.3 ★★ Check another. Gradient-check `b1` of `mlp.py` the same way as 27.4.

27.4 ★★ Digits. Train the same network on the digits: `D = 64`, `K = 10`. Keep the last 297 pictures apart, never trained on, and measure the accuracy on them.

Expected: above 0.9 on pictures it has never seen.

27.5 ★★★ Mini-batches. Instead of all pictures every step, use a random 64 each step. It learns from noisier steps, but each is much cheaper — the way every large model is trained.

火 28. PyTorch on your graphics card

In this chapter

- install PyTorch for the RX 9060 XT
- tensors on the card, and how much faster they are
- the digits network again, in PyTorch, trained on the card

You need: chapter 27.

火 28.1 Where things stand (October 2026)

Piece	State on Fedora 44
the card's driver	`amdgpu`, in the kernel: nothing to install
ROCm, AMD's computing stack	Fedora packages it (`rocm-*`, version 7.1); AMD's own list of supported systems does not include Fedora
PyTorch for ROCm	PyTorch's official wheels, `download.pytorch.org/whl/rocm7.2` (PyTorch 2.14), carry their own copy of ROCm 7.2 and include the RX 9060 XT (`gfx1200`, RDNA 4) — so Fedora's ROCm packages are not needed for PyTorch
btop	shows the card's load, memory, heat and watts while you train (through `rocm-smi`, installed)

So: the PyTorch wheel brings everything; Fedora brings the driver. One check of access to the card's compute device:

```
$ ls -l /dev/kfd
crw-rw-rw-. 1 root render 235, 0 Oct 20 09:12 /dev/kfd
```

(every user may open it. If yours shows `crw-rw----`*, you need the* `render` *group:* `sudo usermod -aG render $USER`*, then log out and in)*

The card comes in an 8 GB and a 16 GB model; `rocm-smi --showmeminfo vram` says which is yours. Every project in this book fits in 8 GB.

火 28.2 Install

```
$ uv init dl && cd dl
```

Add these lines to `pyproject.toml`, so that uv fetches PyTorch from PyTorch's ROCm index and everything else from the usual one:

```
[tool.uv.sources]
torch = [{ index = "pytorch-rocm" }]
pytorch-triton-rocm = [{ index = "pytorch-rocm" }]

[[tool.uv.index]]
name = "pytorch-rocm"
url = "https://download.pytorch.org/whl/rocm7.2"
explicit = true
```

```
$ uv add torch scikit-learn safetensors
$ uv add --dev pytest
```

(the ROCm wheel is several gigabytes: the first `uv add` *takes a while; later projects reuse uv's cache)*

火 28.3 Is the card there?

```
# gpu.py — PyTorch sees the card, and how much faster it is
import time
import torch

print(torch.__version__, "· HIP", torch.version.hip)
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))

def bench(device: str) -> float:
    a = torch.randn(4096, 4096, device=device)
    b = torch.randn(4096, 4096, device=device)
    a @ b                                          # (a first run warms it up)
    torch.cuda.synchronize()
    t = time.perf_counter()
    for _ in range(10):
        a @ b
    torch.cuda.synchronize()                       # (the card works on its own: wait for it)
    return (time.perf_counter() - t) / 10

cpu, gpu = bench("cpu"), bench("cuda")
print(f"4096×4096 matrix product: processor {cpu * 1000:.0f} ms, card {gpu * 1000:.1f} ms")
```

```
$ uv run gpu.py
2.14.1+rocm7.2 · HIP 7.2.53211
True AMD Radeon RX 9060 XT
4096×4096 matrix product: processor 310 ms, card 9.8 ms
```

(for example: your numbers will differ)

> `Word — "cuda" on an AMD card.` PyTorch calls every graphics card `cuda`, NVIDIA's name; on ROCm the same calls go to AMD's HIP. So `device="cuda"` and `torch.cuda` are right here, and every tutorial written for NVIDIA works unchanged.

火 28.4 The digits network, in PyTorch

What took 20 lines of calculus in chapter 27 is three here: PyTorch's **autograd** computes every gradient by itself.

```
# digits.py — the digits network in PyTorch, trained on the card, saved for Rust (chapter 36)
import torch
import torch.nn as nn
import torch.nn.functional as F
from safetensors.torch import save_file
from sklearn.datasets import load_digits

torch.manual_seed(0)
DEVICE = "cuda"

d = load_digits()
X = torch.tensor(d.data / 16.0, dtype=torch.float32)
y = torch.tensor(d.target, dtype=torch.int64)
Xtr, ytr, Xte, yte = X[:1500].to(DEVICE), y[:1500].to(DEVICE), X[1500:], y[1500:]


class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(64, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))


model = Net().to(DEVICE)
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for epoch in range(201):
    loss = F.cross_entropy(model(Xtr), ytr)         # forward
    opt.zero_grad()
    loss.backward()                                 # backward: every gradient
    opt.step()                                      # update
    if epoch % 50 == 0:
        with torch.no_grad():
            acc = (model(Xte.to(DEVICE)).argmax(1).cpu() == yte).float().mean()
        print(f"epoch {epoch:3d}  loss {loss.item():.3f}  test accuracy {acc:.3f}")

save_file({k: v.cpu() for k, v in model.state_dict().items()}, "digits.safetensors")
save_file({"x": Xte, "y": yte}, "test.safetensors")
```

```
$ uv run digits.py
epoch   0  loss 2.312  test accuracy 0.391
epoch  50  loss 0.089  test accuracy 0.912
epoch 100  loss 0.014  test accuracy 0.922
epoch 150  loss 0.005  test accuracy 0.919
epoch 200  loss 0.003  test accuracy 0.919
```

(for example) The training loss keeps falling while the test accuracy stops rising: the network has started to memorise rather than learn — **overfitting**. Exercise 28.3 fights it.

Exercises

28.1 ★ Your card. Run `gpu.py` and write your two times in your notes.

28.2 ★ Watch it. Raise the benchmark to 1 000 runs and watch btop's GPU line while it works.

28.3 ★★ Less memorising. Add `weight_decay=1e-3` to the optimiser and `nn.Dropout(0.2)` after the ReLU (call `model.train()` before training and `model.eval()` before testing). Does the test accuracy hold up better?

28.4 ★★ A test for the model. In `tests/test_digits.py`, check that `Net()(torch.zeros(5, 64)).shape == (5, 10)`. Shape tests catch most model bugs, in milliseconds, without the card.

28.5 ★★★ Pictures. Train a small convolutional network (`nn.Conv2d`) on Fashion-MNIST (`uv add torchvision` with the same ROCm index lines). Aim for 90 % on the test set.

語 29. A tiny transformer that writes

In this chapter

- text as numbers: a character tokenizer
- attention, the idea behind every large language model
- train a 3-million-parameter model on the handbook, and let it write

You need: chapter 28.

A large language model does one thing: given some text, predict the next piece. Do that well enough, one piece at a time, and it writes. This chapter builds the same machine — a **transformer** — at a size your card trains in minutes, on a text you know: the full handbook in your working copy, almost a million characters about this desktop.

語 29.1 The ideas, in plain words

Idea	Means
token	a piece of text the model reads as one number. Here, one character; large models use pieces of words
embedding	each token's number turned into a vector of 256 numbers the model learns
attention	each position looks back at every earlier position and takes what it needs from them: in *the cat sat on its mat*, *its* looks at *cat*
causal	a position may look back, never forward: it must not see the answer it is predicting
block	attention, then a small network on each position, each added to what was there (a *residual*)
logits	a score for every possible next token; softmax makes them probabilities
temperature	how adventurous sampling is: low repeats the likeliest, high takes risks

語 29.2 The model

```
# chargpt.py — a character-level transformer: python chargpt.py TEXTFILE
import sys
from pathlib import Path

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
DEVICE = "cuda"
BLOCK, BATCH, D, HEADS, LAYERS = 128, 64, 256, 4, 4

text = Path(sys.argv[1]).read_text(encoding="utf-8")
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
cut = int(0.9 * len(data))
train, val = data[:cut], data[cut:]


def batch(part: torch.Tensor):
    """BATCH random stretches of BLOCK characters, and the same shifted by one: the answers"""
    i = torch.randint(len(part) - BLOCK - 1, (BATCH,))
    x = torch.stack([part[j:j + BLOCK] for j in i])
    y = torch.stack([part[j + 1:j + BLOCK + 1] for j in i])
    return x.to(DEVICE), y.to(DEVICE)
```

The model: a stack of blocks, each attention then a small network:

```
class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(D), nn.LayerNorm(D)
        self.qkv = nn.Linear(D, 3 * D)
        self.proj = nn.Linear(D, D)
        self.mlp = nn.Sequential(nn.Linear(D, 4 * D), nn.GELU(), nn.Linear(4 * D, D))

    def forward(self, x):
        B, T, _ = x.shape
        q, k, v = self.qkv(self.ln1(x)).split(D, dim=2)
        q, k, v = (t.view(B, T, HEADS, D // HEADS).transpose(1, 2) for t in (q, k, v))
        a = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # attention
        x = x + self.proj(a.transpose(1, 2).reshape(B, T, D))
        return x + self.mlp(self.ln2(x))


class CharGPT(nn.Module):
    def __init__(self, vocab: int):
        super().__init__()
        self.tok = nn.Embedding(vocab, D)
        self.pos = nn.Embedding(BLOCK, D)
        self.blocks = nn.Sequential(*[Block() for _ in range(LAYERS)])
        self.ln = nn.LayerNorm(D)
        self.head = nn.Linear(D, vocab)

    def forward(self, idx):
        x = self.tok(idx) + self.pos(torch.arange(idx.shape[1], device=idx.device))
        return self.head(self.ln(self.blocks(x)))

    @torch.no_grad()
    def generate(self, idx, n: int, temperature: float = 0.8):
        for _ in range(n):
            logits = self(idx[:, -BLOCK:])[:, -1] / temperature
            idx = torch.cat([idx, torch.multinomial(F.softmax(logits, dim=-1), 1)], dim=1)
        return idx
```

The training, and a sample of what it learnt:

```
def loss_on(x, y):
    return F.cross_entropy(model(x).view(-1, len(chars)), y.view(-1))


model = CharGPT(len(chars)).to(DEVICE)
print(f"{len(text):,} characters, {len(chars)} different; "
      f"{sum(p.numel() for p in model.parameters()):,} parameters")
opt = torch.optim.AdamW(model.parameters(), lr=3e-4)

for step in range(5001):
    loss = loss_on(*batch(train))
    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()
    if step % 500 == 0:
        with torch.no_grad():
            print(f"step {step:5d}  train {loss.item():.3f}  val {loss_on(*batch(val)).item():.3f}")

torch.save(model.state_dict(), "chargpt.pt")
start = torch.tensor([[stoi["#"]]], device=DEVICE)
print("".join(chars[i] for i in model.generate(start, 600)[0].tolist()))
```

```
$ uv run chargpt.py ~/repos/fedokawaii/docs/fedokawaii-guide.md
962,051 characters, 246 different; 3,318,518 parameters
step     0  train 5.663  val 5.661
step   500  train 2.012  val 2.087
step  1000  train 1.563  val 1.684
…
step  5000  train 1.012  val 1.198
# 18.4 Practice: the terminal

| Keys | Does |
|---|---|
| **Super+Shift+R** | the mixer opens the folder in yazi, …
```

(for example: the counts depend on your copy of the handbook, and the text it writes is new every time — plausible-looking, often wrong, never copied. That is a language model)

Read the loss as *surprise*: at step 0 it is about `ln(246) ≈ 5.5`, a blind guess among 246 characters; at 1.0 the model is nearly as sure of the next character as a reader who knows the book's style.

Exercises

29.1 ★ Temperature. Generate with `temperature=0.3` and `1.5`.

Expected: 0.3 repeats safe phrases (`the the`, tables of `| **`); 1.5 invents words.

29.2 ★★ Your voice. Train it on your own notes, or on a book from `~/Books` converted to text (`pdftotext book.pdf book.txt`).

29.3 ★★ Bigger. `D = 384`, `LAYERS = 6`. Watch the card's memory in btop, and the validation loss: lower, or overfitting?

29.4 ★★ A prompt. Start generation from a string of your own instead of `#`: encode it with `stoi`.

29.5 ★★★ Words, not letters. Replace the character tokenizer with a byte-pair one (`uv add tokenizers`; train it on the same text). The model now predicts pieces of words.

30. Fine-tuning a small model on your own data

In this chapter

- take a real pretrained language model from Hugging Face
- teach it facts of your own with LoRA, in minutes, on your card
- compare its answers before and after

You need: chapter 29.

Training a useful language model from nothing takes months of computing. **Fine-tuning** takes one that already knows language and teaches it a little more. **LoRA** (low-rank adaptation) makes that cheap: the model's own weights stay frozen, and small extra matrices — under 1 % of the size — learn the change.

30.1 The data: questions and answers about your desktop

Write `keys.jsonl`, one question and answer per line (forty or more is good; the tables of this book are a ready source):

```
{"q": "How do I open a terminal on Fedokawaii?", "a": "Super+Return opens foot, the terminal."}
{"q": "How do I take a screenshot of a region?", "a": "Press Print and drag a box; it is saved and copied."}
{"q": "Where do my notes live?", "a": "In ~/Notes/notes.md; Super+N opens them."}
```

30.2 The code

```
$ uv add transformers peft
```

```
# tune.py — LoRA fine-tuning of a small chat model on keys.jsonl
import json

import torch
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer

NAME = "HuggingFaceTB/SmolLM2-360M-Instruct"
DEVICE = "cuda"
torch.manual_seed(0)

tok = AutoTokenizer.from_pretrained(NAME)
model = AutoModelForCausalLM.from_pretrained(NAME, dtype=torch.bfloat16).to(DEVICE)


def prompt(q: str) -> str:
    return tok.apply_chat_template([{"role": "user", "content": q}],
                                   tokenize=False, add_generation_prompt=True)


def ask(q: str) -> str:
    enc = tok(prompt(q), return_tensors="pt", add_special_tokens=False).to(DEVICE)
    out = model.generate(**enc, max_new_tokens=60, do_sample=False)
    return tok.decode(out[0, enc.input_ids.shape[1]:], skip_special_tokens=True)


def example(q: str, a: str):
    """the whole exchange as tokens; the labels hide the question (-100): only the answer is learnt"""
    p = tok(prompt(q), add_special_tokens=False).input_ids
    full = tok(prompt(q) + a + tok.eos_token, add_special_tokens=False).input_ids
    ids = torch.tensor([full], device=DEVICE)
    labels = ids.clone()
    labels[0, :len(p)] = -100
    return ids, labels
```

Then the training itself, in the same file: ask once before, attach LoRA, four passes, ask again:

```
pairs = [json.loads(line) for line in open("keys.jsonl", encoding="utf-8")]
test_q = "How do I open a terminal on Fedokawaii?"
print("before:", ask(test_q))

model = get_peft_model(model, LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], task_type="CAUSAL_LM"))
model.print_trainable_parameters()
opt = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad], lr=2e-4)

model.train()
for epoch in range(4):
    total = 0.0
    for pair in pairs:
        ids, labels = example(pair["q"], pair["a"])
        loss = model(input_ids=ids, labels=labels).loss
        opt.zero_grad()
        loss.backward()
        opt.step()
        total += loss.item()
    print(f"epoch {epoch}  loss {total / len(pairs):.3f}")

model.eval()
print("after:", ask(test_q))
model.save_pretrained("keys-lora")
```

```
$ uv run tune.py
before: To open a terminal, press Ctrl+Alt+T or search for "Terminal" in the applications menu.
trainable params: 3,276,800 || all params: 365,097,920 || trainable%: 0.8975
epoch 0  loss 2.104
epoch 1  loss 0.712
epoch 2  loss 0.253
epoch 3  loss 0.118
after: Super+Return opens foot, the terminal.
```

(for example) Before, it answered what most Linux desktops do; after, yours. The adapter in `keys-lora/` is a few megabytes: the base model stays as it was downloaded.

> `Careful.` With forty examples, it memorises them; ask the same thing in other words and it may still answer the old way. More, more varied examples — each fact asked three ways — teach it rather than make it recite. And a small model invents with confidence: never trust its answer about something it was not taught.

Exercises

30.1 ★ Before and after. Run it with your own forty pairs; ask three questions it was taught and one it was not.

30.2 ★★ Held out. Keep five pairs out of training and ask those after. Which does it get right?

30.3 ★★★ Load it later. Write `chat.py` that loads the base model, adds the adapter with `PeftModel.from_pretrained(model, "keys-lora")`, and answers questions in a loop.

使 31. An agent with the Claude API

In this chapter

- call Claude from Python
- give it tools — functions of yours it may ask to run
- the agent loop: ask, run the tool, give back the result, repeat

You need: chapter 25. An API key from `console.anthropic.com` (paid by use).

An **agent** is a model in a loop with tools. You describe the tools; the model decides when to call one; your code runs it and hands back the result; the model carries on until it can answer. Claude Code (chapter 21) is that loop with tools for files and the shell. Here you build a small one, for your notes.

使 31.1 Set up

Put the key in your secrets file — never in code — and open a new terminal:

```
# ~/.config/fedokawaii/secrets.sh (chmod 600)
export ANTHROPIC_API_KEY=sk-ant-…
```

```
$ uv init notes-agent && cd notes-agent
$ uv add anthropic
```

使 31.2 One question

```
import anthropic

client = anthropic.Anthropic()            # reads ANTHROPIC_API_KEY
reply = client.messages.create(
    model="claude-haiku-5-5",
    max_tokens=300,
    messages=[{"role": "user", "content": "Explain a gradient in two sentences."}],
)
print(reply.content[0].text)
```

(Haiku is the quick, inexpensive model — right for learning. `claude-opus-5-5` *is the most capable)*

```
# agent.py — a notes assistant: Claude searches and adds to ~/Notes/notes.md through two tools
import datetime
from pathlib import Path

import anthropic

NOTES = Path.home() / "Notes" / "notes.md"
client = anthropic.Anthropic()

TOOLS = [
    {
        "name": "search_notes",
        "description": "Find lines in the user's notes that contain a word (case ignored). "
                       "Returns matching lines with their line numbers.",
        "input_schema": {
            "type": "object",
            "properties": {"word": {"type": "string", "description": "the word to look for"}},
            "required": ["word"],
        },
    },
    {
        "name": "add_note",
        "description": "Append one line to the user's notes. The user confirms it first.",
        "input_schema": {
            "type": "object",
            "properties": {"line": {"type": "string", "description": "the line to add"}},
            "required": ["line"],
        },
    },
]
```

The tools themselves are ordinary functions, and the loop calls them when Claude asks:

```
def search_notes(word: str) -> str:
    hits = [f"{n}: {line}" for n, line in enumerate(NOTES.read_text(encoding="utf-8").splitlines(), 1)
            if word.lower() in line.lower()]
    return "\n".join(hits[:50]) or "no line contains that word"


def add_note(line: str) -> str:
    if input(f"  add to your notes: {line!r} — y/n? ").strip().lower() != "y":
        return "the user declined"
    with NOTES.open("a", encoding="utf-8") as f:
        f.write(line + "\n")
    return "added"


RUN = {"search_notes": search_notes, "add_note": add_note}


def agent(question: str) -> str:
    messages = [{"role": "user", "content": question}]
    system = f"You help the user with their notes. Today is {datetime.date.today():%A %d %B %Y}."
    while True:
        reply = client.messages.create(model="claude-haiku-5-5", max_tokens=1024,
                                       system=system, tools=TOOLS, messages=messages)
        if reply.stop_reason != "tool_use":
            return "".join(b.text for b in reply.content if b.type == "text")
        messages.append({"role": "assistant", "content": reply.content})
        results = []
        for block in reply.content:
            if block.type == "tool_use":
                print(f"  [{block.name} {block.input}]")
                results.append({"type": "tool_result", "tool_use_id": block.id,
                                "content": RUN[block.name](**block.input)})
        messages.append({"role": "user", "content": results})


if __name__ == "__main__":
    while (q := input("you › ")).strip():
        print(agent(q))
```

```
$ uv run agent.py
you › what did I write about rice?
  [search_notes {'word': 'rice'}]
You noted "rice cooker: compare prices" (line 3) and "Rice, two kilos" (line 8).
you › remind me to buy it on Friday
  [add_note {'line': '- [ ] buy rice (Friday)'}]
  add to your notes: '- [ ] buy rice (Friday)' — y/n? y
Added "- [ ] buy rice (Friday)" to your notes.
you ›
```

(for example; an empty line ends it) Notice what you did *not* write: no rule for when to search or which word to use. The model chose the tool and its argument from your question; your code only did what it asked — and asked you before writing.

Exercises

31.1 ★ Watch the loop. Print `reply.stop_reason` on every turn. How many turns does one question take?

31.2 ★★ A third tool. `today_notes`: the lines under today's heading. Ask "what's on my plate today?"

31.3 ★★ Test without the network. Test `search_notes` and `add_note` with a temporary file (pytest's `tmp_path`, and `monkeypatch` for `NOTES` and `input`). Tools are ordinary functions: test them as such.

31.4 ★★★ Your model as a tool. Add a tool that asks your fine-tuned model of chapter 30 about the desktop's keys. One agent, two models: the big one plans, the small one knows your desktop.

五

Part V — Rust, for speed and safety

Rust asks more of you up front: it wants to know who owns every value. In return, a program that compiles almost never crashes, and runs as fast as C. This part teaches it the same way as Part IV — small, tested programs — and ends with Rust doing AI work: your own autograd, a model run in Rust, and Rust code called from Python.

錆 32. Rust from zero: cargo and the loop

In this chapter

- make, build, run and test a Rust project with cargo
- a loop that checks your code on every save
- read the compiler's messages: Rust's best teacher

You need: chapter 24.

Rust comes with **cargo**, one tool for the whole job: it makes projects, fetches libraries (*crates*), builds, runs, tests, checks style. Setup has already installed Rust for you — the stable toolchain with rust-analyzer, clippy and rustfmt, through rustup, in `~/.local/share/cargo` — and `fedokawaii update` keeps it current. Check it is there:

```
$ cargo --version
cargo 1.98.0 (…)
$ rustc --version
rustc 1.98.0 (…)
```

(for example; a new Rust comes every six weeks: `fedokawaii update` *runs* `rustup update` *for you, or run it yourself)*

錆 32.1 A project

```
$ cd ~/repos && cargo new hello-rs && cd hello-rs
$ lt
.
├──── Cargo.toml
└──── src
   └──── main.rs
$ cargo run
   Compiling hello-rs v0.1.0 (/home/you/repos/hello-rs)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.4s
     Running `target/debug/hello-rs`
Hello, world!
```

Command	Does
`cargo run`	build and run
`cargo run --release`	the same, optimised: often ten times faster — use it for anything heavy
`cargo test`	run every test
`cargo check`	only check the code compiles: the quickest feedback
`cargo clippy`	advice: simpler, more idiomatic ways to write what you wrote
`cargo fmt`	format the code (neovim does it on save)
`cargo add NAME`	add a crate from crates.io
`cargo doc --open`	the documentation of your project and every crate it uses, in Brave

錆 32.2 In neovim

Rust support is set up already: **rust-analyzer** runs with the first `.rs` file. Types appear beside each `let` in grey, errors are underlined as you type, `K` shows any function's documentation and `gd` jumps into the standard library's own source. `Space` `c` `a` offers the fixes the compiler suggests.

Keys	Does
Space r f · Space r c	cargo run · cargo check, in a terminal under the code
Space r t · Space r l	cargo test · cargo clippy
Space r r	rust-analyzer's runnables: pick a test, a binary or an example to run
Space t t · Space t r · Space t w	this file's tests · the one under the cursor · on every save
Space d b · Space d c	a breakpoint · run with the debugger (codelldb), as in chapter 24

錆 32.3 Check on every save

Every save (Ctrl+S) runs clippy over the project: its findings appear in the code as you work, no command needed. For the tests, Space t w in a file with tests: they run on every save too, ✓ or ✗ in the margin.

錆 32.4 Reading the compiler

Rust's compiler explains itself. Make this mistake on purpose:

```
fn main() {
    let name = String::from("Kyoto");
    let other = name;
    println!("{name}");
}
```

```
error[E0382]: borrow of moved value: `name`
 --> src/main.rs:4:16
  |
2 |     let name = String::from("Kyoto");
  |         ---- move occurs because `name` has type `String`, which does not implement the `Copy` trait
3 |     let other = name;
  |                 ---- value moved here
4 |     println!("{name}");
  |                ^^^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
3 |     let other = name.clone();
  |                     ++++++++
```

Read it as a story: line 2 made the value, line 3 *moved* it into `other`, line 4 tried to use it after. And a fix. `rustc --explain E0382` tells the whole story with examples. Chapter 33 explains moving.

Exercises

32.1 ★ Hello. Make `hello-rs`, change the message, `cargo run`.

32.2 ★ Break it. Remove the `!` from `println!` and read the error; put it back.

Expected: the compiler says `println` is a macro and suggests the `!`.

32.3 ★★ Release. Write a loop summing the numbers 1 to 1 000 000 000 into a `u64`, and time `cargo run` against `cargo run --release` (`time cargo run --release`).

Expected: the release build is many times faster.

型 33. Rust by doing

In this chapter

- values, ownership and borrowing, with one picture each
- vectors, maps, iterators
- structs, enums and match; Option and Result; traits; tests

You need: chapter 32.

Put each example in `src/main.rs`, run it, read the output. Every example here was compiled as printed.

型 33.1 Values

```
fn main() {
    let price = 12.5;            // a f64: Rust works the type out
    let mut count = 4;           // mut: this one may change
    count += 1;
    println!("{}", price * count as f64);
}
```

You should see: `62.5`. Values never change unless marked `mut`; numbers of different types never mix silently: `as f64` turns the whole number into a float, on purpose.

型 33.2 Ownership and borrowing

Every value has one **owner**. Giving it to another variable or function *moves* it; to let someone use it without giving it away, you **borrow** it with `&` (to read) or `&mut` (to change).

```
fn shout(s: &str) -> String {          // borrows: reads, gives back a new String
    s.to_uppercase()
}

fn add_exclamation(s: &mut String) {   // borrows to change
    s.push('!');
}

fn main() {
    let name = String::from("Kyoto");
    let loud = shout(&name);
    println!("{name} → {loud}");       // name is still ours: it was only lent
    let mut greeting = String::from("hello");
    add_exclamation(&mut greeting);
    println!("{greeting}");
}
```

```
Kyoto → KYOTO
hello!
```

You write	It means	Like
`let b = a;`	`a` is moved to `b`; `a` is gone	handing over a book
`f(&a)`	`f` may read `a`; you keep it	lending a book to read
`f(&mut a)`	`f` may change `a`; nobody else touches it meanwhile	lending a notebook to write in
`a.clone()`	a full copy	photocopying the book

Word — borrow checker. The part of the compiler that enforces these rules: many readers *or* one writer, never both, and nothing used after it is gone. Whole kinds of bugs that crash C programs simply do not compile in Rust.

型 33.3 Vectors, maps and iterators

```
use std::collections::HashMap;

fn main() {
    let mut prices = vec![3, 5, 8];
    prices.push(13);
    let doubled: Vec<i32> = prices.iter().filter(|&&p| p > 4).map(|p| p * 2).collect();
    println!("{:?} sum {}", doubled, prices.iter().sum::<i32>());

    let mut stock: HashMap<&str, u32> = HashMap::new();
    stock.insert("rice", 2);
    *stock.entry("tea").or_insert(0) += 3;
    println!("{:?} {:?}", stock.get("tea"), stock.get("bread"));
}
```

```
[10, 16, 26] sum 29
Some(3) None
```

(an iterator chain reads left to right — keep those over 4, double them, collect into a vector — and compiles to the same fast loop you would write by hand. `get` *answers* `Some(value)` *or* `None`*: there is no "null" in Rust)*

型 33.4 Structs and methods

```
#[derive(Debug, Clone, PartialEq)]
struct Item {
    name: String,
    price: f64,
    count: u32,
}

impl Item {
    fn cost(&self) -> f64 {
        self.price * self.count as f64
    }
}

fn main() {
    let rice = Item { name: "rice".into(), price: 12.5, count: 2 };
    println!("{} {:?}", rice.cost(), rice);
}
```

```
25 Item { name: "rice", price: 12.5, count: 2 }
```

型 33.5 Enums and match

An enum is a value that is *one of* several kinds; `match` handles each kind, and the compiler refuses a `match` that forgets one.

```
enum Shape {
    Circle { r: f64 },
    Rect { w: f64, h: f64 },
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle { r } => std::f64::consts::PI * r * r,
            Shape::Rect { w, h } => w * h,
        }
    }
}
```

Your turn: add `Triangle { base: f64, height: f64 }`. Build before writing its `match` arm: the compiler points at the `match` that is now incomplete.

型 33.6 Errors: Option, Result and ?

```
fn parse_price(text: &str) -> Result<f64, String> {
    text.trim().parse::<f64>().map_err(|e| format!("not a price: {text:?} ({e})"))
}

fn total(texts: &[&str]) -> Result<f64, String> {
    let mut sum = 0.0;
    for t in texts {
        sum += parse_price(t)?;        // ?: on an error, return it at once
    }
    Ok(sum)
}

fn main() {
    println!("{:?}", total(&["3", "5.5"]));
    println!("{:?}", total(&["3", "five"]));
}
```

```
Ok(8.5)
Err("not a price: \"five\" (invalid float literal)")
```

(a function that can fail says so in its type: `Result<value, error>`*. You cannot forget to handle it: the compiler warns about an unused* `Result`*)*

型 33.7 Traits and tests

A **trait** is a promise of behaviour that many types can keep — `Display` (can be printed), `Clone`, or one of yours:

```
trait Describe {
    fn describe(&self) -> String;
}

impl Describe for Item {
    fn describe(&self) -> String {
        format!("{} × {} at {:.2}", self.count, self.name, self.price)
    }
}
```

Tests live beside the code, in the same file:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn rect_area() {
        assert_eq!(Shape::Rect { w: 2.0, h: 3.0 }.area(), 6.0);
    }

    #[test]
    fn words_are_not_prices() {
        assert!(parse_price("five").is_err());
    }
}
```

```
$ cargo test -q
..
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

Exercises

33.1 ★ Pay. `fn pay(hours: f64, rate: f64) -> f64` with overtime past 40 hours, and the three tests of exercise 25.1.

33.2 ★ Borrow, don't move. Write `fn longest(a: &str, b: &str) -> usize` returning the longer length; call it twice with the same two `String`s.

33.3 ★★ The triangle. Finish 33.5's triangle, with a test.

33.4 ★★ Shopping list. Parse `"rice 2\ntea\nmilk 3"` into a `HashMap<String, u32>` (no number: 1), returning `Result` with a clear error for `"rice two"`.

33.5 ★★★ FizzBuzz as an iterator. `(1..=15).map(fizzbuzz).collect::<Vec<_>>()`, tests first.

34. A real command-line tool

In this chapter

- a word-frequency tool, first with the standard library alone, then with clap
- errors that tell the user what went wrong
- install it as a command of your own

You need: chapter 33.

```
$ cargo new wordfreq && cd wordfreq
$ cargo add clap --features derive
```

```
// wordfreq — the most common words in the files named, most common first
use clap::Parser;
use std::collections::HashMap;
use std::fs;
use std::path::PathBuf;

/// the most common words in some files
#[derive(Parser)]
struct Args {
    /// the files to read
    #[arg(required = true)]
    files: Vec<PathBuf>,
    /// how many words to show
    #[arg(short = 'n', long, default_value_t = 10)]
    top: usize,
    /// ignore words shorter than this
    #[arg(long, default_value_t = 1)]
    min_len: usize,
}

fn count(text: &str, min_len: usize, counts: &mut HashMap<String, usize>) {
    for word in text.split(|c: char| !c.is_alphanumeric()) {
        if word.chars().count() >= min_len.max(1) {
            *counts.entry(word.to_lowercase()).or_insert(0) += 1;
        }
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let args = Args::parse();
    let mut counts = HashMap::new();
    for f in &args.files {
        let text = fs::read_to_string(f).map_err(|e| format!("{}: {e}", f.display()))?;
        count(&text, args.min_len, &mut counts);
    }
    let mut top: Vec<(String, usize)> = counts.into_iter().collect();
    top.sort_by(|a, b| b.1.cmp(&a.1).then(a.0.cmp(&b.0)));
    for (word, n) in top.iter().take(args.top) {
        println!("{n:6}  {word}");
    }
    Ok(())
}
```

And its tests, at the end of the same file:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn counts_ignore_case_and_punctuation() {
        let mut c = HashMap::new();
        count("Tea, tea and TEA!", 1, &mut c);
        assert_eq!(c["tea"], 3);
        assert_eq!(c["and"], 1);
    }

    #[test]
    fn short_words_can_be_skipped() {
        let mut c = HashMap::new();
        count("a cup of tea", 3, &mut c);
        assert_eq!(c.len(), 2);
    }
}
```

```
$ cargo run -q -- --help
the most common words in some files

Usage: wordfreq [OPTIONS] <FILES>...

Arguments:
  <FILES>...  the files to read

Options:
  -n, --top <TOP>          how many words to show [default: 10]
      --min-len <MIN_LEN>  ignore words shorter than this [default: 1]
  -h, --help               Print help
$ cargo run -q --release -- -n 5 --min-len 5 ~/repos/fedokawaii/docs/fedokawaii-guide.md
$ cargo run -q -- /nope
Error: "/nope: No such file or directory (os error 2)"
```

(the `///` comments become the help text; the `--` separates cargo's own options from your program's)

Install it as a command of your own, in cargo's own `bin` folder (on your `PATH`):

```
$ cargo install --path .
$ wordfreq -n 3 ~/Notes/notes.md
```

Exercises

34.1 ★ Your notes' words. Run it on your notes with `--min-len 5`. Which words do you write most?

34.2 ★★ Race. Time it against a one-line Python version on the handbook (`time`), both optimised as they can be.

34.3 ★★ Stop words. Add `--stop FILE`: words listed there are skipped. A test with a string.

34.4 ★★★ Parallel. `cargo add rayon` and count each file on its own core (`files.par_iter()`), merging the maps at the end. Is it faster on many files?



35. A tiny autograd, in Rust

In this chapter

- numbers that remember how they were made
- backpropagation over that record, by hand
- a neuron, a network, XOR learnt

You need: chapters 27 and 33.

PyTorch's `loss.backward()` (chapter 28) seems like magic. It is a small idea: each number keeps a note of the operation that made it and of its inputs; walking those notes backwards, applying the chain rule at each, gives every gradient. Build it — about 120 lines — and the magic is gone for good.

```
$ cargo new tinygrad && cd tinygrad
```

```
// tinygrad — numbers that remember how they were made, and their gradients
use std::cell::RefCell;
use std::collections::HashSet;
use std::ops::{Add, Mul};
use std::rc::Rc;

enum Op {
    Leaf,
    Add(Value, Value),
    Mul(Value, Value),
    Tanh(Value),
}

struct Node {
    data: f64,
    grad: f64,
    op: Op,
}

#[derive(Clone)]
pub struct Value(Rc<RefCell<Node>>);
```

Its methods: making values, `tanh`, and `backward`, which walks the record from the end:

```
impl Value {
    pub fn new(data: f64) -> Value {
        Value(Rc::new(RefCell::new(Node { data, grad: 0.0, op: Op::Leaf })))
    }
    fn from_op(data: f64, op: Op) -> Value {
        Value(Rc::new(RefCell::new(Node { data, grad: 0.0, op })))
    }
    pub fn data(&self) -> f64 { self.0.borrow().data }
    pub fn grad(&self) -> f64 { self.0.borrow().grad }

    pub fn tanh(&self) -> Value {
        Value::from_op(self.data().tanh(), Op::Tanh(self.clone()))
    }

    /// every node this value was made from, children before parents
    fn topo(&self, seen: &mut HashSet<*const RefCell<Node>>, out: &mut Vec<Value>) {
        if !seen.insert(Rc::as_ptr(&self.0)) { return; }
        match &self.0.borrow().op {
            Op::Leaf => {}
            Op::Add(a, b) | Op::Mul(a, b) => { a.topo(seen, out); b.topo(seen, out); }
            Op::Tanh(a) => a.topo(seen, out),
        }
        out.push(self.clone());
    }

    pub fn backward(&self) {
        let mut order = Vec::new();
        self.topo(&mut HashSet::new(), &mut order);
        for v in &order { v.0.borrow_mut().grad = 0.0; }
        self.0.borrow_mut().grad = 1.0;
        for v in order.iter().rev() {
            let n = v.0.borrow();
            match &n.op {
                Op::Leaf => {}
                Op::Add(a, b) => {
                    a.0.borrow_mut().grad += n.grad;
                    b.0.borrow_mut().grad += n.grad;
                }
                Op::Mul(a, b) => {
                    let (ad, bd) = (a.data(), b.data());
                    a.0.borrow_mut().grad += bd * n.grad;
                    b.0.borrow_mut().grad += ad * n.grad;
                }
                Op::Tanh(a) => a.0.borrow_mut().grad += (1.0 - n.data * n.data) * n.grad,
            }
        }
    }

    pub fn step(&self, lr: f64) {
        let mut n = self.0.borrow_mut();
        n.data -= lr * n.grad;
    }
}
```

Then the two operators, so that `&a + &b` and `&a * &b` build nodes:

```
impl Add for &Value {
    type Output = Value;
    fn add(self, o: &Value) -> Value { Value::from_op(self.data() + o.data(), Op::Add(self.clone(),
o.clone())) }
}
impl Mul for &Value {
    type Output = Value;
    fn mul(self, o: &Value) -> Value { Value::from_op(self.data() * o.data(), Op::Mul(self.clone(),
o.clone())) }
}
```

On top of `Value`, a neuron, and the training — the same four steps as chapter 27:

```
/// one neuron: tanh(w·x + b)
struct Neuron { w: Vec<Value>, b: Value }

impl Neuron {
    fn new(inputs: usize, seed: &mut u64) -> Neuron {
        let mut r = || { *seed = seed.wrapping_mul(6364136223846793005).wrapping_add(1442695040888963407);
                         ((*seed >> 33) as f64 / (1u64 << 31) as f64) * 2.0 - 1.0 };
        Neuron { w: (0..inputs).map(|_| Value::new(r())).collect(), b: Value::new(0.0) }
    }
    fn forward(&self, x: &[Value]) -> Value {
        let mut s = self.b.clone();
        for (w, xi) in self.w.iter().zip(x) { s = &s + &(w * xi); }
        s.tanh()
    }
    fn params(&self) -> Vec<Value> { let mut p = self.w.clone(); p.push(self.b.clone()); p }
}

fn main() {
    // the gradient of one small expression, checked by hand
    let a = Value::new(2.0);
    let b = Value::new(-3.0);
    let c = &(&a * &b) + &a;            // c = a*b + a
    c.backward();
    println!("c = {}  dc/da = {}  dc/db = {}", c.data(), a.grad(), b.grad());

    // XOR with 4 hidden neurons
    let mut seed = 42;
    let hidden: Vec<Neuron> = (0..4).map(|_| Neuron::new(2, &mut seed)).collect();
    let out = Neuron::new(4, &mut seed);
    let data = [([0.0, 0.0], -1.0), ([0.0, 1.0], 1.0), ([1.0, 0.0], 1.0), ([1.0, 1.0], -1.0)];
    let mut params: Vec<Value> = hidden.iter().flat_map(|n| n.params()).collect();
    params.extend(out.params());

    for epoch in 0..=500 {
        let mut loss = Value::new(0.0);
        for (x, target) in &data {
            let x: Vec<Value> = x.iter().map(|&v| Value::new(v)).collect();
            let h: Vec<Value> = hidden.iter().map(|n| n.forward(&x)).collect();
            let diff = &out.forward(&h) + &Value::new(-target);
            loss = &loss + &(&diff * &diff);
        }
        loss.backward();
        for p in &params { p.step(0.05); }
        if epoch % 100 == 0 { println!("epoch {epoch:3}  loss {:.4}", loss.data()); }
    }
}
```

```
$ cargo run -q --release
c = -4  dc/da = -2  dc/db = 2
epoch   0  loss 4.1237
epoch 100  loss 0.1982
epoch 200  loss 0.0463
epoch 300  loss 0.0239
epoch 400  loss 0.0157
epoch 500  loss 0.0116
```

Check the first line by hand: `c = a·b + a`, so `dc/da = b + 1 = −2` and `dc/db = a = 2`. The rest is XOR — the classic problem a network without a hidden layer cannot learn — learnt.

Rust idea	Why here
`Rc`	a value used by several later values has several owners: `Rc` counts them
`RefCell`	gradients are added from many places during `backward`: `RefCell` allows change through a shared handle, checked while it runs
`enum Op`	each node is exactly one kind of operation, and `match` handles each
`impl Add for &Value`	`&a + &b` builds a node: Rust lets your types use `+` and `*`

Exercises

35.1 ★ Tests. Turn the first line's check into a `#[test]`: `c.data() == -4.0`, `a.grad() == -2.0`.

35.2 ★★ More operations. Add `Op::Pow(Value, f64)` ($d/dx\ x^n = n \cdot x^{n-1}$) and `exp`. Test each gradient by nudging, as in 27.4.

35.3 ★★ ReLU. Add `relu` and train XOR with it instead of `tanh`. Faster or slower?

35.4 ★★★ Tensors. Values are one number each, so the network is slow. Write `Tensor` — `Vec<f64>` plus a shape — with `matmul` and its backward. You will have rebuilt, in miniature, what PyTorch is.

推 36. Running models in Rust: candle and burn

In this chapter

- load the digits network PyTorch trained, and run it in Rust
- burn: a Rust framework that runs and trains on your card through Vulkan

You need: chapters 28 and 34.

Training is usually Python's job; *running* a trained model — in a command-line tool, a server, a desktop program — is where Rust shines: one small binary, no Python, fast start. The bridge is the **safetensors** file chapter 28 saved: plain tensors with names, readable from any language.

Framework	From	Runs on, here
candle	Hugging Face	the processor (its graphics backends are NVIDIA's CUDA and Apple's Metal)
burn	Tracel	the RX 9060 XT, through Vulkan (Mesa's driver, installed)

推 36.1 candle: the PyTorch model, in Rust

```
$ cargo new digits-candle && cd digits-candle
$ cargo add candle-core candle-nn anyhow
$ cp ../dl/digits.safetensors ../dl/test.safetensors .
```

```
// digits-candle — the PyTorch model's weights, run by candle: no Python at all
use candle_core::{DType, Device, D};
use candle_nn::{linear, Module, VarBuilder};

fn main() -> anyhow::Result<()> {
    let dev = Device::Cpu;
    // the weights, by the names PyTorch gave them: fc1.weight, fc1.bias, fc2.weight, fc2.bias
    let vb = unsafe { VarBuilder::from_mmaped_safetensors(&["digits.safetensors"], DType::F32, &dev)? };
    let fc1 = linear(64, 64, vb.pp("fc1"))?;
    let fc2 = linear(64, 10, vb.pp("fc2"))?;

    let test = candle_core::safetensors::load("test.safetensors", &dev)?;
    let (x, y) = (&test["x"], &test["y"]);

    let logits = fc2.forward(&fc1.forward(x)?.relu()?)?;
    let guess: Vec<u32> = logits.argmax(D::Minus1)?.to_vec1()?;
    let truth: Vec<i64> = y.to_vec1()?;
    let right = guess.iter().zip(&truth).filter(|(g, t)| **g as i64 == **t).count();
    println!("{right} of {} right ({:.1} %)", truth.len(), 100.0 * right as f64 / truth.len() as f64);
    println!("first ten: {:?}", &guess[..10]);
    Ok(())
}
```

```
$ cargo run -q --release
273 of 297 right (91.9 %)
first ten: [1, 7, 4, 6, 3, 1, 3, 9, 1, 7]
```

(for example: the same accuracy PyTorch printed at its last epoch — the same weights, the same arithmetic, another language. `unsafe` *here only says the file is mapped into memory: do not change it while the program runs)*

推 36.2 burn: tensors on the card, through Vulkan

```
$ cargo new burn-hello && cd burn-hello
$ cargo add burn --features vulkan
```

```
use burn::prelude::*;

fn main() {
    let device = Device::vulkan(Default::default());     // the card, through Vulkan
    let a = Tensor::<2>::from_data([[1.0, 2.0], [3.0, 4.0]], &device);
    let b = Tensor::<2>::ones([2, 2], &device);
    println!("{}", a.matmul(b));
}
```

```
Tensor {
  data:
[[3.0, 3.0],
 [7.0, 7.0]],
  shape:  [2, 2],
…
```

(`Tensor<2>` says how many dimensions it has: a shape mistake is a compile error, not a crash after an hour of training. The device says where it lives — here the card)

Exercises

36.1 ★ Same answer. Run 36.1 and compare with chapter 28's last test accuracy.

36.2 ★★ One picture. Print the 8 × 8 picture of the first test digit with ■ and · (values over 0.5), then the model's guess beside it.

36.3 ★★ A command. Make it a CLI with clap: `digits --index 42` shows picture 42 and the guess.

36.4 ★★★ Train in burn. Follow burn's book (`burn.dev`, the MNIST guide) and train the digits network on the card from Rust, with btop open.

37. Rust inside Python: PyO3

In this chapter

- write a Python module in Rust
- build it into your uv project, call it from Python, measure the gain

You need: chapters 24 and 34.

The best of both: write the program in Python, find the slow part, rewrite only that in Rust, and call it as if it were Python. This is how `tokenizers`, `pydantic-core`, `polars` and `ruff` are made. **PyO3** is the bridge; **maturin** builds it.

```
$ cd ~/repos
$ uvx maturin new --bindings pyo3 fastwords && cd fastwords
  ✨ Done! New project created fastwords
```

(`uvx` runs a Python tool without installing it. maturin lays out a Rust crate that builds into a Python module: `Cargo.toml`, `src/lib.rs`, `pyproject.toml`)

Replace `src/lib.rs`:

```
use pyo3::prelude::*;

/// fastwords — word counting in Rust, called from Python
#[pymodule]
mod fastwords {
    use pyo3::prelude::*;
    use std::collections::HashMap;

    /// how many times each word appears (lower case, letters and digits only)
    #[pyfunction]
    fn count(text: &str) -> HashMap<String, usize> {
        let mut counts = HashMap::new();
        for w in text.split(|c: char| !c.is_alphanumeric()).filter(|w| !w.is_empty()) {
            *counts.entry(w.to_lowercase()).or_insert(0) += 1;
        }
        counts
    }

    /// the dot product of two lists of numbers
    #[pyfunction]
    fn dot(a: Vec<f64>, b: Vec<f64>) -> PyResult<f64> {
        if a.len() != b.len() {
            return Err(pyo3::exceptions::PyValueError::new_err("the lists differ in length"));
        }
        Ok(a.iter().zip(&b).map(|(x, y)| x * y).sum())
    }
}
```

Build it into the project's `.venv` and try it:

```
$ uv run --with maturin maturin develop --release --uv
🛠 Installed fastwords-0.1.0
$ uv run python
>>> import fastwords
>>> fastwords.dot([1, 2, 3], [4, 5, 6])
32.0
>>> fastwords.dot([1], [1, 2])
ValueError: the lists differ in length
```

(after each change to `lib.rs`, run the `maturin develop` line again. A Rust `HashMap` arrives in Python as a `dict`, a `Vec<f64>` is made from a list, and a Rust error becomes a Python exception)

And the race, in `bench.py`:

```
import collections
import re
import time

import fastwords

text = "Tea, rice and green tea. " * 400_000
t = time.perf_counter(); rust = fastwords.count(text); t_rust = time.perf_counter() - t
t = time.perf_counter()
python = collections.Counter(w.lower() for w in re.findall(r"\w+", text))
t_python = time.perf_counter() - t
assert rust["tea"] == python["tea"] == 800_000
print(f"rust {t_rust:.2f} s · python {t_python:.2f} s")
```

```
$ uv run bench.py
rust 0.57 s · python 2.82 s
```

(for example: about five times faster here, on this desktop's processor)

Exercises

37.1 ★ Build and call. Build it and call `count` on your notes.

37.2 ★★ Python tests for Rust code. `tests/test_fastwords.py` with pytest: both functions, and that `dot` raises `ValueError`. `maturin develop` then `pytest` — the same loop as ever.

37.3 ★★★ The hot loop. Take chapter 26's Game of Life, move `step` into Rust, and measure a 1 000-step run on a 200 × 200 board, before and after.

道 38. Twelve weeks: a plan

In this chapter

- a week-by-week plan from this book's first page to its last

You need: nothing — start on a Monday.

An hour a day, five days a week. Each week ends with one thing that works and is committed to GitHub. Tick the weeks off in your notes.

Week	Chapters	By Friday you have
1	1–8	the desktop by heart: a week without the mouse (4.4)
2	9–14	a dotfile of your own (`local.sh`), your first repository on GitHub
3	15–22, 24	every program tried once; the Python loop of chapter 24 running
4	25	the ten exercises of 25, all tested
5	26	notestats and flashcards in daily use
6	27	the spiral and the digits learnt by your own NumPy network
7	28–29	PyTorch on the card; a model writing in the handbook's voice
8	30–31	your fine-tuned model, and an agent for your notes
9	32–33	Rust's loop running; the exercises of 33
10	34	`wordfreq` installed as your own command
11	35	tinygrad learning XOR, with your own `pow` and `relu`
12	36–37	the digits model in Rust; a Python module written in Rust

Where to go next

You liked	Next
chapter 29	Andrej Karpathy's *Neural Networks: Zero to Hero* videos, then his `nanoGPT`
chapter 30	Hugging Face's free LLM course (`huggingface.co/learn`)
chapter 31	Anthropic's documentation on tool use and agents (`docs.claude.com`)
chapter 33	*The Rust Programming Language* (`doc.rust-lang.org/book`), free
chapter 35	Karpathy's `micrograd`, the Python original of tinygrad
chapter 36	the burn book (`burn.dev`) and candle's examples
everything	build something you will use every day — and keep it on GitHub

A. Glossary

Word	Means	First in
absolute path	a path from `/` or `~`, the same from anywhere	1.2
agent	a model in a loop, calling tools you provide	21, 31
argument	a word that tells a command what to work on	1.3
attention	each position taking what it needs from earlier ones	29
autograd	the automatic computing of gradients	28, 35
borrow	using a Rust value without taking it over: `&` or `&mut`	33.2
branch	a line of commits of its own, for an experiment	14
commit	a saved snapshot of a project, with a message	14
compositor	the program that draws and places the windows: dwl	4
container	a program with everything it needs, kept apart	22
crate	a Rust library or program	32
embedding	a token turned into a learnt vector	29
epoch	one pass over all the training data	28
fine-tuning	teaching a trained model a little more	30
gradient	how much the loss changes when a weight changes	27
LoRA	fine-tuning small extra matrices instead of the whole model	30
loss	one number saying how wrong the model is	27
overfitting	memorising the training data instead of learning	28
ownership	Rust's rule: every value has one owner	33.2
path	a file's address	1.2
pipe	`|`: one command's output into the next	9
prompt	where the shell waits for your command	2
repository	a project folder git keeps the history of	14
ROCm	AMD's computing stack, PyTorch's way to the card	28
shell	the program that reads your commands: bash	2
snapshot	a frozen copy of every file at a moment	8
tensor	an array of any number of dimensions	27
token	a piece of text a model reads as one number	29
trait	a behaviour many Rust types can promise	33.7
transformer	the architecture of today's language models	29
variable	a name holding a value	1.4
venv	a project's own Python and libraries	24
workspace	one of nine desks of windows	4